Fakultät Informatik an der Technischen Hochschule Nürnberg



Bachelorarbeit

My very fancy thesis title

Georg Simon Ohm
Matrikelnummer: 1234567

Prüfer: Prof. Dr. Max Mustermann
Prof. Dr. Monika Musterfrau

Betreuer: Prof. Dr. Max Musterbetreuer
Prof. Dr. Monika Musterbetreuerin
Example AG

© Example AG 2026

Diese Arbeit, einschließlich ihrer Teile, **ist urheberrechtlich geschützt**. Jede Verwertung außerhalb der engen Grenzen des Urheberrechtsgesetzes ohne Zustimmung von Example AG ist untersagt und strafbar. Dies gilt insbesondere für Vervielfältigungen, Übersetzungen, Mikroverfilmung und die Speicherung und Verarbeitung in elektronischen Systemen.

Danksagung

Only if you want to thank someone.

Kurzdarstellung

Kurze Zusammenfassung der Arbeit, höchstens eine halbe Seite. Deutsche Fassung auch nötig, wenn die Arbeit auf Englisch angefertigt wird. Das Template warnt vor falschen Angaben.

Keywords
Container Image, Software Supply Chain Security

Abstract

Only if thesis is written in English.

Inhaltsverzeichnis

1 Einleitung **1**
1.1 Motivation und Relevanz ... 1
1.2 Problemstellung ... 1
1.3 Fallstudie und Zielsetzung ... 2
1.4 Forschungsfrage ... 2
1.5 Aufbau der Arbeit ... 2

2 Theoretischer Hintergrund **4**

3 Methodisches Vorgehen **6**
3.1 Forschungsansatz und Erkenntnisziel ... 6
3.2 Untersuchungsgegenstand und Analyseverfahren ... 6
3.3 Ablauf der Entwicklung ... 7
3.3.1 Phase 1: Überführung in eine lokal nutzbare Offline-Anwendung ... 7
3.3.2 Phase 2: Ergänzung um Synchronisierung und weitere Online-Anbindungen ... 7
3.3.3 Phase 3: Konsolidierte Prüfung und Härtung ... 7
3.4 Ableitung des Leitfadens und Bewertung ... 8

4 Analyse und Anforderungsdefinition **9**
4.1 Vom übergeordneten Ziel zur Fallauswahl ... 9
4.2 Funktionen und Architektur des Ausgangsartefakts ... 10
4.2.1 Funktionaler Untersuchungsumfang ... 10
4.2.2 Technische Komponenten und Repository-Struktur ... 10
4.2.3 Lokale Funktionen und exemplarische Online-Anbindungen ... 13
4.3 Zielarchitektur und Sicherheitsbegriff ... 14
4.4 Anforderungskatalog ... 15
4.4.1 A1: Lauffähige Desktop-Anwendung ... 15
4.4.2 A2: Dreischichtenarchitektur ... 15
4.4.3 A3: Begrenzung des Refactoring-Aufwands ... 15
4.4.4 A4: Lokale Kommunikation ohne HTTP-Wrapper ... 15
4.4.5 A5: Lokale Datenhaltung und Offline-Kernfunktionen ... 15
4.4.6 A6: Local-First-Kollaboration am Gruppenfeature „Aufgaben“ ... 16
4.4.7 A7: Härtung gemäß Electron Security Guidelines ... 16
4.4.8 A8: Repräsentativer Kernfunktionsumfang und Erweiterbarkeit ... 16
4.4.9 A9: Nachvollziehbarer Migrationsleitfaden ... 16
4.5 Abgrenzung des Untersuchungsumfangs ... 16

5 Entwurf, Migration und Migrationsleitfaden **18**
5.1 Struktur des Migrationsleitfadens ... 18

5.2 Phase 1: Von der Webanwendung zur Offline-Kernanwendung 19
5.2.1 Initialisierung des Electron-Projekts und Übernahme der Datenbankmodule 19
5.2.2 Anpassung der Tabellen und Datenbankabhängigkeiten an SQLite 19
5.2.3 Versionierte Schemamigrationen und Datenbankinitialisierung 21
5.2.4 Übernahme des API-Vertrags und Anpassung der Codegenerierung 22
5.2.5 IPC-Transport, Preload-Schnittstelle und Dispatcher 23
5.2.6 Begründung und Grenzen der vertragsbasierten Generierung 24
5.2.7 Übernahme der React-Oberfläche in den Renderer 26
5.2.8 Anbindung der Seiten, Datumswerte und Navigation 27
5.2.9 Lokale Authentifizierung als erster vollständiger Verarbeitungspfad 28
5.2.10 Integration der CRUD-Funktionen am Beispiel der Notenverwaltung 28
5.2.11 Lokaler Dateiimport am Beispiel iCal 30
5.2.12 Abschluss der Offline-Kernanwendung und Übergang zu Phase 2 31
5.3 Phase 2: Netzwerkanbindungen und Local-First-Synchronisierung 31
5.3.1 Auswahlkriterien und betrachtete Synchronisierungslösungen 31
5.3.2 Entscheidung für PowerSync und Supabase 32
5.3.3 Vorprüfung der Integration und Grenzen der Wiederverwendung 33
5.3.4 Architektur mit zwei lokalen Datenhaltungsbereichen 33
5.3.5 Technische Integration der Kollaborationsmodule 35
5.3.6 Anpassung des Einladungsablaufs und vorausgesetzte Konten 37
5.3.7 Schrittweise Überführung weiterer Funktionen 38
5.3.8 Verantwortung für Schemaänderungen und Updates 39
5.3.9 Grenzen der untersuchten Synchronisierung 40
5.3.10 Online-Abruf des Studienkatalogs 40
5.4 Phase 3: Prüfung und Härtung der vervollständigten Anwendung 41
5.5 Ausarbeitung der einzelnen Migrationsschritte 41

6 Evaluation und Demonstration **42**

7 Diskussion **45**

8 Fazit und Ausblick **47**

A Ursprüngliche Erstellungs-Prompts **56**
A.1 Prompt 1: Persönliche Studienorganisation 56
A.2 Prompt 2: Studiengruppen und Zusammenarbeit 58

Abkürzungsverzeichnis

Kapitel 1

Einleitung

1.1 Motivation und Relevanz

Viele Menschen möchten Anwendungen für den persönlichen Gebrauch erstellen, verfügen jedoch über keine Programmiererfahrung. Durch KI-gestützte Werkzeuge wie Replit oder Lovable ist es ihnen heute möglich, innerhalb kurzer Zeit funktionsfähige Anwendungen zu erzeugen, ohne eine einzige Zeile Code selbst zu schreiben oder zu verstehen. Diese Anwendungen nutzen die Erstellenden im Vertrauen darauf, sie seien sicher und produktionsreif. Tatsächlich betrachten sie den erzeugten Code jedoch nie; sie beurteilen die Anwendung ausschließlich anhand ihrer sichtbaren Funktionen. Der Quellcode bleibt für sie eine Blackbox, deren Qualität und Sicherheit sie nicht einschätzen können.

Die Relevanz dieser Entwicklung reicht über die Erstellung kleiner Hobby-Werkzeuge hinaus: KI-gestützte Entwicklungsplattformen richten sich auch an Unternehmen und werben mit Anwendungen für den produktiven Einsatz, wie etwa die Unternehmensbeispiele auf der Website von Lovable zeigen. [1] Damit gewinnen Anforderungen an Sicherheit, Zuverlässigkeit und Wartbarkeit an Bedeutung. Werden solche Anwendungen zudem als lokale Desktop-Anwendungen mit direktem Zugriff auf Dateisystem und Systemressourcen betrieben, müssen zusätzliche Sicherheitsanforderungen berücksichtigt werden. Eine Anwendung, die oberflächlich funktioniert, aber unbemerkt unsicher oder architektonisch mangelhaft ist, kann in einer solchen lokalen Laufzeitumgebung Schäden verursachen, die über die Zugriffsmöglichkeiten einer im Browser isolierten Anwendung hinausgehen. Genau an dieser Stelle setzt die vorliegende Arbeit an.

1.2 Problemstellung

Von technischen Laien instruierte generative KI-Modelle fokussieren sich nahezu ausschließlich auf die Umsetzung der gewünschten Funktionen. Fundamentale Qualitätsmerkmale des Software Engineerings nach ISO/IEC 25010 — Wartbarkeit, Trennung der Belange, Interoperabilität und Sicherheit — bleiben dabei regelmäßig unberücksichtigt. Auch die Produktdarstellung von Lovable betont die schnelle Umsetzung beschriebener Ideen und deren unmittelbare Bereitstellung. Zwar bewirbt die Plattform zugleich Sicherheitsfunktionen; aus der sichtbaren Funktionalität und den Produktversprechen allein lässt sich jedoch nicht ableiten, ob der erzeugte Code die Qualitätsanforderungen eines konkreten Einsatzszenarios erfüllt. [1] Der entstehende Code ist häufig unstrukturiert und stark gekoppelt, was spätere Wartung und Erweiterung erschwert. Da die Erstellenden nur die Funktionen, nicht aber den Code beurteilen, bleibt diese Diskrepanz für sie unsichtbar: Die Anwendung wirkt einsatzbereit, obwohl sie es nicht ist.

Besonders kritisch wird diese Diskrepanz, sobald eine solche Webanwendung in eine lokale Desktop-Umgebung überführt werden soll. Die Zielumgebung stellt völlig andere Anforderungen als die isolierte Sandbox des KI-Werkzeugs, und die vom Werkzeug getroffenen Architekturentscheidungen lassen sich

nicht unverändert übernehmen. Technisch wäre es mit vergleichsweise geringem Aufwand möglich, eine Webanwendung nur oberflächlich zu verpacken, etwa indem im Main-Prozess einer Electron-Anwendung eine lokale Web-API betrieben und das Frontend unverändert im Renderer ausgeliefert wird. Ein solcher Ansatz ermöglicht zwar die Ausführung auf dem Desktop, berücksichtigt für sich genommen jedoch nicht die zusätzlichen Sicherheitsanforderungen, die durch den Zugriff auf lokale Dateien und Systemressourcen entstehen. Ohne eine gezielte Anpassung der Vertrauensgrenzen und Zugriffsmöglichkeiten können Schwachstellen der ursprünglichen Webanwendung dadurch Auswirkungen auf das lokale System erhalten. Ziel der Arbeit ist deshalb nicht die bloße Ausführbarkeit auf dem Desktop, sondern eine sichere, architektonisch saubere Transformation. Für genau diese Klasse von Anwendungen fehlt bislang ein systematisches Vorgehen, um eine von Laien in Auftrag gegebene, KI-generierte Webanwendung sicher in eine lokale Desktop-Anwendung zu transformieren.

1.3 Fallstudie und Zielsetzung

Um diese Lücke zu schließen, entwickelt die vorliegende Arbeit einen methodischen Migrationspfad für die Überführung KI-generierter Webanwendungen in sichere Local-First-Desktop-Anwendungen. Validierungsobjekt ist eine konkrete Fallstudie: der Studienplaner, eine deutschsprachige Studienverwaltungsanwendung, die als Replit-Webprototyp vorliegt und in eine Electron-Anwendung überführt wird. Entscheidend ist dabei, wie dieser Prototyp entstanden ist: Er wurde bewusst so in Auftrag gegeben, wie es eine Person ohne technischen Hintergrund tun würde. Die beiden ursprünglichen Prompts beschreiben in überwiegend alltagssprachlicher Form gewünschte Funktionen und Nutzungsszenarien, etwa einen persönlichen Stundenplan und die Zusammenarbeit in Studiengruppen (Abschnitt A.1 und Abschnitt A.2). Sie enthalten auch Anforderungen an Anmeldung und Zugriffsschutz, machen jedoch keine Vorgaben zur Wahl eines UI-Frameworks oder einer konkreten Backend- oder Datenbanktechnologie. Der Studienplaner ist damit keine von einem erfahrenen Entwickler gestaltete Anwendung, sondern eine authentisch laienhaft erzeugte, und repräsentiert so genau die Klasse von Artefakten, auf die der Migrationspfad abzielt.

Die Wahl des Local-First-Paradigmas als Zielarchitektur ist dabei nicht beliebig. Local-First bündelt mehrere für die Migration KI-generierter Software zentrale Eigenschaften: Offline-Fähigkeit, Datensouveränität und eine vom Netzwerk entkoppelte, dauerhaft verfügbare Nutzung. Die Arbeit untersucht jedoch nicht sämtliche Aspekte dieses Paradigmas, sondern setzt einen besonderen Schwerpunkt auf die Synchronisierung lokal gespeicherter Daten. Damit soll veranschaulicht werden, dass die Migration in eine Desktop-Umgebung nicht auf den Umbau zu einer ausschließlich offline nutzbaren Anwendung beschränkt ist. Online-Funktionen können erhalten bleiben und mit lokaler Datenhaltung sowie einer auch ohne Netzwerk verfügbaren Kernfunktionalität verbunden werden.

1.4 Forschungsfrage

FF1: Wie können KI-generierte Webarchitekturen mithilfe eines framework-agnostischen Ansatzes systematisch in sichere Local-First-Desktop-Anwendungen überführt werden?

1.5 Aufbau der Arbeit

Der weitere Aufbau der Arbeit orientiert sich am Design-Science-Research-Prozess und gliedert sich in sieben weitere Kapitel. Kapitel 2 legt den theoretischen Hintergrund, indem es KI-gestützte Entwicklungswerkzeuge, die Besonderheiten von Web-zu-Desktop-Architekturen, das Local-First-Paradigma sowie sicherheitsrelevante Grundlagen einordnet und die bestehende Forschungslücke herausarbeitet. Kapitel 3 stellt den methodischen Rahmen der Arbeit vor, begründet die Wahl des Design-Science-Research-Ansatzes und skizziert den eigenen Forschungsprozess. Kapitel 4 widmet sich der Problemanalyse: Zunächst wird die Softwarelandschaft KI-gestützter Entwicklungswerkzeuge, etwa Ansätze wie Replit oder Lovable, betrachtet, bevor in die Tiefe des gewählten Ansatzes, der Replit-Umgebung und des daraus resultierenden Artefakts, eingestiegen wird. Kapitel 5 behandelt Entwurf

und Entwicklung des Artefakts, also die konzeptionelle Architektur und die praktische Umsetzung der Electron-Anwendung. Kapitel 6 stellt die Evaluation und Demonstration des entwickelten Migrationspfads anhand der Fallstudie in den Mittelpunkt. Kapitel 7 diskutiert die gewonnenen Erkenntnisse, reflektiert die verfolgten Lösungswege und ordnet die Ergebnisse im Vergleich zu bestehenden Ansätzen ein. Kapitel 8 fasst die zentralen Erkenntnisse zusammen und gibt einen Ausblick auf offene Fragestellungen und zukünftigen Forschungsbedarf.

Kapitel 2

Theoretischer Hintergrund

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine metu degendae praesidia firmissima. – Filium morte multavit. – Si sine causa, nollem me ab eo delectari, quod ista Platonis, Aristoteli, Theophrasti orationis ornamenta neglexerit. Nam illud quidem physici, credere aliquid esse minimum, quod profecto numquam putavisset, si a Polyaeno, familiari suo, geometrica discere maluisset quam illum etiam ipsum dedocere. Sol Democrito magnus videtur, quippe homini erudito in geometriaque perfecto, huic pedalis fortasse; tantum enim esse omnino in nostris poetis aut inertissimae segnitiae est aut fastidii delicatissimi. Mihi quidem videtur,

inermis ac nudus est. Tollit definitiones, nihil de dividendo ac partiendo docet, non quo ignorare vos arbitrer, sed ut ratione et via.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine.

Data are based in the repository[1]. The data is stored in the `data` directory, and the code for processing the data is stored in the `code` directory.

Tabelle 2.1 shows an example dataset that we will use for our experiments.

Name	**Value**	**Unit**
Temperature	23.5	°C
Pressure	1013	hPa
Humidity	45	%

Tabelle 2.1: Example Dataset

[1]https://github.com/th-nuernberg/thesis-template-typst

Kapitel 3

Methodisches Vorgehen

3.1 Forschungsansatz und Erkenntnisziel

Die Arbeit verfolgt einen gestaltungsorientierten Forschungsansatz. Im Mittelpunkt stehen die Entwicklung eines Migrationsleitfadens und dessen Anwendung auf den KI-generierten Webprototyp Studienplaner. Als Forschungsrahmen dient Design Science Research. Das Prozessmodell nach Peffers et al. unterscheidet Problemidentifikation und Motivation, Definition der Lösungsziele, Entwurf und Entwicklung, Demonstration, Evaluation sowie Kommunikation. [2]

Die Zuordnung zu diesem Rahmen strukturiert die wissenschaftliche Darstellung des tatsächlich durchgeführten Vorgehens. Sie bedeutet nicht, dass sämtliche technischen Schritte bereits zu Beginn festgelegt waren. Die Migration wurde schrittweise entwickelt; Erkenntnisse aus der Analyse und Umsetzung führten zu Anpassungen der Architektur und des weiteren Vorgehens. Die nachfolgend beschriebenen Phasen fassen diesen Prozess rückblickend zusammen.

Das Erkenntnisziel besteht darin, die für die Migration notwendigen Veränderungen nachvollziehbar zu beschreiben und daraus einen fallbezogenen Leitfaden abzuleiten. Die Electron-Anwendung dient als technischer Demonstrator. Der Leitfaden dokumentiert die Voraussetzungen, Entscheidungen, Transformationsschritte und Kontrollpunkte der Migration. Aussagen über seine Übertragbarkeit werden auf den untersuchten Fall begrenzt.

3.2 Untersuchungsgegenstand und Analyseverfahren

Ausgangspunkt war der auf Replit erzeugte Studienplaner. Wie in Kapitel 1 erläutert, erfolgte dessen Erstellung anhand funktionaler Beschreibungen ohne verbindliche Vorgaben zur technischen Architektur. Untersucht wurde damit ein KI-generiertes Ausgangsartefakt, dessen Komponenten für eine lokale Desktop-Laufzeit neu eingeordnet werden mussten.

Die beiden ursprünglichen Erstellungs-Prompts sind in Abschnitt A.1 und Abschnitt A.2 vollständig dokumentiert. Sie beschreiben in überwiegend alltagssprachlicher Form zunächst die persönliche Studienorganisation und anschließend die Zusammenarbeit in Studiengruppen. Neben funktionalen Wünschen nennen sie Anforderungen an Anmeldung und Zugriffsschutz, legen jedoch weder ein UI-Framework noch eine konkrete Backend- oder Datenbanktechnologie fest.

Die Untersuchung erfolgte anhand des Quellcodes, der Projektkonfiguration und der vorhandenen technischen Dokumentation. Dabei wurden Oberfläche, Schnittstellen, Geschäftslogik, Datenhaltung und externe Abhängigkeiten erfasst. Für die Migration war insbesondere zu klären, welche Funktionen unmittelbar von einem Backend oder einer Netzwerkverbindung abhingen, welche Bestandteile lokal weiterverwendet werden konnten und an welchen Stellen eine Anpassung der Verantwortlichkeiten erforderlich war.

Ergänzend wurde die offizielle Plattformdokumentation herangezogen, um die Wahl des Untersuchungsgegenstands einzuordnen. Replit und Lovable wurden hierzu beispielhaft hinsichtlich ihrer dokumentierten Backend-Ansätze betrachtet. Diese Betrachtung dient der Begründung des Untersuchungsumfangs; sie ist keine empirische Vergleichsstudie der Plattformen. Die Ergebnisse und der daraus abgeleitete Anforderungskatalog werden in Kapitel 4 dargestellt.

3.3 Ablauf der Entwicklung

Die technische Entwicklung gliederte sich in drei aufeinander aufbauende Phasen. Diese sind Arbeitsschritte innerhalb der Entwurfs- und Entwicklungsaktivität des Forschungsrahmens und bilden kein eigenständiges Forschungsmodell.

3.3.1 Phase 1: Überführung in eine lokal nutzbare Offline-Anwendung

Zunächst wurden die eingeschlossenen Kernfunktionen von der ursprünglichen Server-Laufzeit entkoppelt. Hierzu wurden die Komponenten auf eine Desktop-Architektur mit getrennten Verantwortlichkeiten für Darstellung, Geschäftslogik und Datenzugriff abgebildet. Die Datenhaltung wurde lokal bereitgestellt. Aufrufe der bisherigen Webschnittstellen wurden in einen begrenzten Kommunikationsweg zwischen Renderer, Preload und Main-Prozess überführt.

Ziel dieser Phase war die Entwicklung einer Electron-Anwendung, deren lokale Kernfunktionen ohne Netzwerkverbindung ausführbar sind und deren Architektur zugleich die Grundlage für die nachfolgenden Erweiterungen bildet. Die Verantwortlichkeiten und Schnittstellen wurden so gestaltet, dass Synchronisierung und weitere externe Dienste später integriert werden können, ohne die lokale Kernarchitektur erneut grundlegend umzustrukturieren. Damit verbindet diese Phase die Herstellung lokaler Nutzbarkeit mit der Vorbereitung einer schrittweise erweiterbaren Anwendung.

3.3.2 Phase 2: Ergänzung um Synchronisierung und weitere Online-Anbindungen

Aufbauend auf der lokal ausführbaren Kernanwendung wurden in der zweiten Phase die netzwerkabhängigen Funktionen betrachtet. Diese waren bei der Herstellung der Offline-Kernanwendung zunächst ausgeklammert worden. Ziel war es, externe Datenquellen und Dienste schrittweise in die vorbereitete Desktop-Architektur einzubinden und dadurch den angestrebten Funktionsumfang zu vervollständigen.

Die Wiederanbindung wurde anhand zweier unterschiedlich komplexer Fälle untersucht. Der Abruf eines Studiengangkatalogs dient als Beispiel für das Herunterladen externer Daten und deren Übernahme in die lokale Anwendung. Dieser Vorgang umfasst keine Synchronisierung und veranschaulicht zunächst einen einfachen Netzwerkzugriff. Darauf aufbauend wurde mit der gemeinsamen Aufgabenverwaltung in Studiengruppen ein Kollaborationsfall betrachtet. Hier werden lokal verarbeitete Änderungen über eine zusätzliche Synchronisierungsinfrastruktur zwischen den Beteiligten ausgetauscht. Dieser zweite Fall greift gezielt Eigenschaften des Local-First-Paradigmas auf.

3.3.3 Phase 3: Konsolidierte Prüfung und Härtung

Nach der funktionalen Umsetzung wurden die entstandenen lokalen und netzwerkbezogenen Schnittstellen systematisch hinsichtlich der Electron-Sicherheitsrichtlinien geprüft und erforderliche Anpassungen vorgenommen. Der Sicherheitsbegriff wird für diese Arbeit über die Umsetzung der anwendbaren Electron Security Guidelines einschließlich Eingabevalidierung und Maßnahmen gegen die Ausführung fremden Codes operationalisiert. [3]

Diese abschließende Phase bündelt die Prüfung der fertigen Anwendung. Sicherheitsrelevante Architekturentscheidungen, insbesondere die Trennung privilegierter Funktionen vom Renderer und die Begrenzung der Preload-Schnittstellen, betreffen bereits die vorausgehenden Phasen. Die Darstellung

einer abschließenden Härtung bedeutet daher nicht, dass Sicherheit erst nach Abschluss der Entwicklung berücksichtigt wird.

3.4 Ableitung des Leitfadens und Bewertung

Aus dem dokumentierten Migrationsprozess werden die Schritte des Leitfadens abgeleitet. Für jeden Schritt werden Ausgangszustand, Ziel, notwendige Änderungen und ein Kontrollpunkt beschrieben. Dabei werden architektonische Prinzipien von Anpassungen unterschieden, die an den konkreten Technologieeinsatz der Fallstudie gebunden sind. Die technischen Schritte und ihre Zuordnung zum Demonstrator werden in Kapitel 5 erläutert.

Die Bewertung orientiert sich am Anforderungskatalog aus Kapitel 4. Architektur- und Quellcodeinspektionen, vorhandene Service-Unit-Tests sowie begrenzte manuelle Demonstrationen bilden die vorgesehenen Nachweisformen. Für den Sicherheitsnachweis werden Anwendbarkeit, Umsetzung und Beleg jeder Electron-Empfehlung getrennt erfasst. Bereits vorliegende Sicherheitsberichte werden mit ihrem jeweiligen Prüfstand und ihren Nachweisgrenzen berücksichtigt.

Die Festlegung der Nachweise ist von den Ergebnissen ihrer Durchführung zu unterscheiden. Aus der Beschreibung des Verfahrens folgt keine pauschale Erfüllung der Anforderungen. Die Ergebnisdarstellung erfolgt in Kapitel 6. Ein weiterer Migrationsdurchlauf zur unabhängigen Erprobung des Leitfadens ist nicht Bestandteil der Arbeit. Die Bewertung stützt sich auf den dokumentierten Prozess und den vorhandenen Demonstrator; eine empirisch bestätigte Übertragbarkeit auf weitere Plattformen wird daraus nicht abgeleitet.

Kapitel 4

Analyse und Anforderungsdefinition

4.1 Vom übergeordneten Ziel zur Fallauswahl

Die in Kapitel 1 formulierte Forschungsfrage nimmt einen frameworkagnostischen Migrationsansatz als Ausgangsziel auf. Für dessen Bearbeitung ist zunächst zu bestimmen, welche Architekturmerkmale des Ausgangsartefakts die Transformation beeinflussen. Entscheidend ist nicht allein das verwendete Frontend-Framework, sondern auch, wo Geschäftslogik, Datenhaltung und Authentifizierung ausgeführt werden und welche dieser Bestandteile im verfügbaren Quellcode enthalten sind.

Zur Einordnung werden Replit und Lovable anhand ihrer offiziellen Dokumentation beispielhaft betrachtet. Replit beschreibt Webanwendungen als standardmäßig erzeugte Full-Stack-Anwendungen mit Frontend und Backend. API-Routen, Datenbank und serverseitige Logik werden nach dem Bedarf der Anwendung eingerichtet. Diese Dokumentation beschreibt das Plattformangebot; den konkreten Aufbau des Studienplaners belegt erst die Analyse des vorliegenden Artefakts. [4]

Lovable dokumentiert dagegen einen standardmäßig aktivierten verwalteten Cloud-Backend-Dienst auf Grundlage der Open-Source-Technologie von Supabase. Alternativ kann ein eigenes Supabase-Projekt angebunden werden. Datenbank, Authentifizierung, Dateispeicherung und serverseitige Funktionen werden damit als Backend-Dienste bereitgestellt. Diese dokumentierte Dienststruktur wird hier als Backend-as-a-Service eingeordnet. Daraus folgt jedoch nicht, dass jedes Lovable-Projekt dieselbe Architektur aufweist. [5]

Für die Migration ergibt sich daraus eine unterschiedliche Ausgangslage. Liegt die serverseitige Anwendungslogik als eigener Quellcode vor, kann ihre Übernahme und Anpassung an die lokale Laufzeit untersucht werden. Bei einer dienstgebundenen Architektur müssen dagegen auch die vom Dienst übernommenen Verantwortlichkeiten identifiziert und gegebenenfalls lokal nachgebildet werden. Eine reine Ersetzung des Frontend-Transports deckt diese zweite Ausgangslage nicht vollständig ab.

Die Entwicklung und Prüfung eines allgemeinen Leitfadens für beide Fälle würde den Umfang dieser Arbeit überschreiten. Die praktische Untersuchung wird daher auf den Replit-Prototyp Studienplaner begrenzt. Lovable dient zur dokumentationsgestützten Begründung dieser Eingrenzung; eine Lovable-Anwendung wird nicht migriert. Der resultierende Leitfaden kann potenziell übertragbare Prinzipien aufzeigen, wird jedoch nicht als validierte Lösung für beliebige Frameworks oder Plattformen ausgegeben.

4.2 Funktionen und Architektur des Ausgangsartefakts

4.2.1 Funktionaler Untersuchungsumfang

Der funktionale Ausgangspunkt ergibt sich aus den ursprünglichen Erstellungs-Prompts zur persönlichen Studienorganisation und zur Zusammenarbeit in Studiengruppen. Beide Prompts sind in Anhang A dokumentiert. Für die Migration werden sämtliche persönlichen Funktionsbereiche sowie Authentifizierung und ausgewählte Gruppenfunktionen betrachtet. Tabelle 4.1 konkretisiert diesen Untersuchungsumfang. Die Aufstellung bildet den funktionalen Bezugsrahmen; maßgeblich für den Demonstrator ist der in Abschnitt 4.3 definierte repräsentative Kernfunktionsumfang.

Bereich	**Betrachtete Funktionen**
Dashboard	Notendurchschnitt, gesammelte ECTS-Punkte, anstehende Fristen, Schnellzugriffe sowie letzte Aktivitäten und nächste Termine.
Kalender	Monatsansicht mit Tages-Popover, Termindetails und Löschen, farbliche Kennzeichnung der Terminarten, iCal-/ICS-Import, mehrtägige Termine und nach Priorität filterbare Listenansicht.
Noten	Semesterweise gruppierte Kurs-, Noten- und ECTS-Tabelle; Anlegen, Lesen, Bearbeiten und Löschen; CSV-Import mit Spaltenerkennung; ECTS-gewichteter Notendurchschnitt und Prüfung der Kurszugehörigkeit zum Studiengang.
Stundenplan	Raster mit fünf Wochentagen und sechs festen Hochschulzeitblöcken, Auswahl vorbereiteter Hochschulpläne, Hinzufügen und Entfernen von Kursen, visuelle Unterscheidung vorgeschlagener und eigener Einträge sowie Leeren des Plans mit Bestätigung.
Authentifizierung	Registrierung, Anmeldung und Abmeldung sowie die Zuordnung persönlicher Daten zur jeweiligen Identität.
Studiengruppen	Anlegen benannter Gruppen und Einladen weiterer Mitglieder; gemeinsame Aufgaben mit Fälligkeit, zuständiger Person und Status „offen“ beziehungsweise „erledigt“.

Tabelle 4.1: Funktionaler Untersuchungsumfang auf Grundlage der ursprünglichen Erstellungs-Prompts.

Im Webprototyp wird die angemeldete Sitzung über Session-Cookies vermittelt. Bei der Migration sind die lokale Zuordnung persönlicher Daten und die für die Zusammenarbeit benötigte entfernte Authentifizierung gesondert zu betrachten. Die konkrete Umsetzung der Identitäts- und Sitzungsverwaltung wird in Kapitel 5 beschrieben.

Der Kollaborationsumfang wird auf Gruppenerstellung, Einladungen und gemeinsame Aufgaben begrenzt. Kommentare, gemeinsame Kalendertermine sowie zusätzliche Gruppeninhalte wie Links und Notizen werden nicht untersucht. Die Sichtbarkeit gemeinsamer Aufgaben bleibt auf die jeweiligen Gruppenmitglieder begrenzt; persönliche Noten und Termine werden durch die Gruppenzugehörigkeit nicht automatisch geteilt.

4.2.2 Technische Komponenten und Repository-Struktur

Der vorliegende Replit-Prototyp enthält eine React-Oberfläche, einen eigenständigen Express-API-Server, einen durch OpenAPI beschriebenen API-Vertrag sowie eine über Drizzle angebundene

PostgreSQL-Datenhaltung. Oberfläche, Backend und gemeinsam genutzte Bibliotheken sind im Projekt getrennt abgelegt. Die Datenbankanbindung erwartet eine konfigurierte entfernte Datenbankverbindung. Damit liegt im untersuchten Fall serverseitiger Quellcode vor, der für die Migration analysiert und angepasst werden kann. [6]

Die Schnittstellen zwischen Frontend und Backend werden in der Datei `lib/api-spec/openapi.yaml` als gemeinsamer API-Vertrag beschrieben. Dieser legt die Endpunkte sowie die Strukturen der Anfragen und Antworten fest. Auf dieser Grundlage erzeugt Orval API-Aufrufhilfen, React-Query-Hooks und Typdefinitionen für das Frontend sowie Zod-Schemata für die Validierung im Backend. Die generierten Bestandteile werden über gemeinsam genutzte Projektbibliotheken eingebunden. Der OpenAPI-Vertrag bildet damit die gemeinsame Grundlage für die Schnittstellendefinition auf beiden Seiten und wird bei der späteren Anpassung des Kommunikationswegs erneut aufgegriffen. [6]

Die Oberfläche in `artifacts/studienplaner/` verwendet React und Vite sowie Tailwind CSS und shadcn/ui für die Darstellung. Der Express-Server liegt in `artifacts/api-server/` und wird mit esbuild als ESM-Anwendung gebündelt. Die gemeinsam genutzten Bibliotheken sind unter `lib/` zusammengefasst: `db/` enthält das Drizzle-Schema und den PostgreSQL-Client, `api-spec/` den OpenAPI-Vertrag, `api-zod/` die daraus erzeugten Validierungsschemata und `api-client-react/` die mit Orval generierten React-Query-Aufrufhilfen. `artifacts/mockup-sandbox/` dient der Gestaltung und Erprobung von Oberflächen und wird nicht als fachlicher Bestandteil der Migration betrachtet. [6]

Abbildung 4.1 stellt die Komponenten und ihre Beziehungen dar. Die durchgezogenen Pfeile zeigen den Kommunikationsweg zur Laufzeit: Die React-Oberfläche ruft den Express-API-Server über HTTP auf; dessen Route-Handler greifen über Drizzle und den PostgreSQL-Client auf die Datenbank zu. Die gestrichelten Pfeile zeigen die Codegenerierung und Einbindung der aus dem OpenAPI-Vertrag erzeugten Bestandteile. Der Vertrag selbst ist damit eine Grundlage der Entwicklung und kein zusätzlicher Vermittler im Laufzeitpfad.

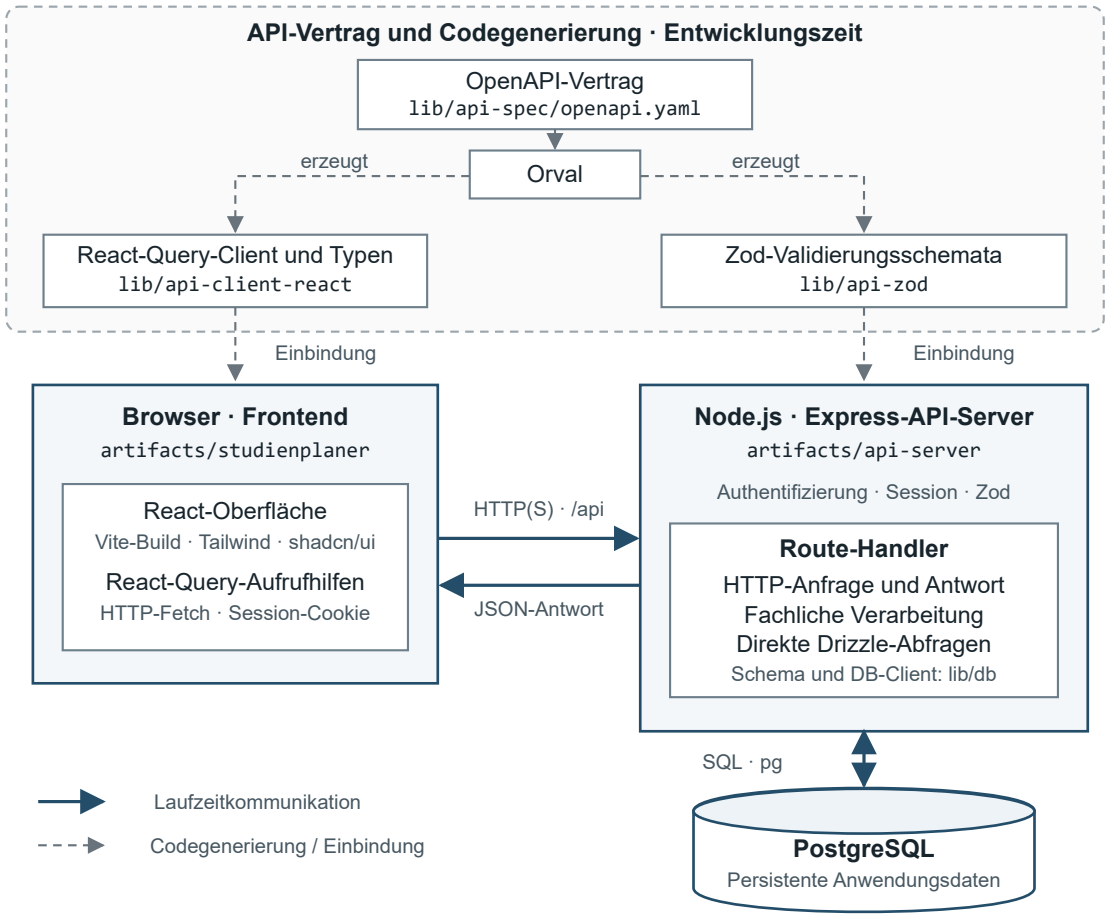


Abbildung 4.1: Ursprüngliche Webarchitektur des Studienplaners mit Laufzeitkommunikation und OpenAPI-basierter Codegenerierung; eigene Darstellung auf Grundlage des Ausgangsartefakts.

Diese Projektaufteilung ist noch kein Nachweis einer durchgängig getrennten Dreischichtenarchitektur. Für die Transformation sind die tatsächlichen Verantwortlichkeiten der Module und ihre Abhängigkeiten maßgeblich. Das in Quellcode 4.1 gezeigte Beispiel veranschaulicht das im API-Server verwendete Verarbeitungsmuster innerhalb eines Route-Handlers.

```
router.get("/dashboard/upcoming", requireAuth, async (req, res): Promise<void> => {
  const userId = req.session.userId!;
  const now = new Date();
  const in7Days = new Date(now.getTime() + 7 * 24 * 60 * 60 * 1000);

  const events = await db
    .select()
    .from(eventsTable)
    .where(
      and(
        eq(eventsTable.userId, userId),
        gte(eventsTable.startDate, now),
        lte(eventsTable.startDate, in7Days),
      ),
    )
    .orderBy(eventsTable.startDate);

  res.json(events);
});
```

Quellcode 4.1: Exemplarischer Express-Route-Handler für die Abfrage anstehender Termine im Ausgangsartefakt.

Der Handler liest nach der Authentifizierungsprüfung die Benutzerkennung aus der Sitzung, selektiert Termine für die kommenden sieben Tage und liefert das Abfrageergebnis als JSON zurück. Damit verbindet er HTTP-Verarbeitung, Fachlogik und Datenzugriff. Die Migration erfordert neben der Entkopplung vom HTTP-Transport eine architektonische Trennung in Controller (Schnittstellenverarbeitung), Service (Fachlogik) und Repository (Datenzugriff). Diese Aufteilung ermöglicht eine isolierte Prüfung der Geschäftslogik mit austauschbaren Datenzugriffen und erleichtert die Wartung sowie Anpassungen an Transport und Persistenz.

Das zentrale Migrationsproblem besteht somit in der Verlagerung der bisher servergebundenen Funktionen in die lokale Desktop-Laufzeit bei gleichzeitigem Erhalt der eingeschlossenen Kernfunktionen. Die Oberfläche soll möglichst weitgehend weiterverwendet werden. Technisch notwendige Anpassungen betreffen insbesondere den Aufrufweg zur Geschäftslogik, das Datenbankschema, die Sitzungsverwaltung und die Behandlung externer Dienste.

4.2.3 Lokale Funktionen und exemplarische Online-Anbindungen

Die persönlichen Funktionen ermöglichen die Untersuchung unterschiedlicher lokaler Interaktionsformen. Die CRUD-Funktionen der Noten-, Termin- und Stundenplanverwaltung stehen für die Verarbeitung von Benutzereingaben und den Zugriff auf persistente Anwendungsdaten. Die CSV- und iCal-/ICS-Importe ergänzen diesen Umfang um die Übernahme dateibasierter Daten. Dabei müssen externe Daten eingelesen, interpretiert, validiert und auf das interne Datenmodell abgebildet werden. Der Untersuchungsumfang umfasst somit sowohl die unmittelbare Bearbeitung von Daten über die Oberfläche als auch deren Übernahme aus Dateien. Diese Interaktionsformen bilden die Grundlage für die lokale Migration, bevor zusätzliche Netzwerkzugriffe und Synchronisierung betrachtet werden.

Für die Zielanwendung werden zwei Online-Anbindungen betrachtet: der Abruf des Studiengangkatalogs und die Zusammenarbeit im ausgewählten Gruppenbereich. Die übrigen eingeschlossenen Funktionen werden so eingeordnet, dass ihre Datenhaltung und Verarbeitung lokal erfolgen können. Dazu zählen auch die persönliche Datenzuordnung und die lokalen Dateiimporte. Nach einem Katalogabruf stehen die übernommenen Daten der lokalen Verarbeitung zur Verfügung; der Abruf selbst benötigt eine Verbindung.

Der Katalog dient als Beispiel für das Herunterladen, Prüfen und lokale Übernehmen externer Daten. Der Gruppenbereich ergänzt dieses Muster um entfernte Authentifizierung, Mitgliedschaft und Einladungen sowie die Synchronisierung lokal verarbeiteter Aufgabenänderungen. Innerhalb dieses Bereichs ist zwischen synchronisierbaren Aufgabenoperationen und unmittelbar netzwerkabhängigen Verwaltungsoperationen zu unterscheiden.

Diese Auswahl deckt zwei für den Leitfaden relevante Integrationsmuster ab: externe Datenübernahme und Local-First-Kollaboration. Weitere Funktionen mit vergleichbaren Anforderungen können auf denselben architektonischen Prinzipien aufbauen. Daraus ergibt sich die Annahme, dass solche Erweiterungen überwiegend eine erneute Anwendung der beschriebenen Muster erfordern. Eine Übertragbarkeit auf Funktionen mit zusätzlichen Anforderungen wird dadurch nicht nachgewiesen; diese wäre im jeweiligen Fall gesondert zu prüfen.

4.3 Zielarchitektur und Sicherheitsbegriff

Ziel der Migration ist eine architektonisch nachvollziehbare Desktop-Anwendung, deren Struktur zugleich die Grundlage des Migrationsleitfadens bildet. Architektonische Qualität wird dabei durch klar zugeordnete Verantwortlichkeiten, definierte Schnittstellen und kontrollierte Abhängigkeiten konkretisiert. Wiederkehrende Aufgaben sollen nach einheitlichen Mustern umgesetzt werden. Dadurch können die Transformationsschritte verständlich beschrieben und auf weitere Funktionen mit vergleichbaren Anforderungen bezogen werden.

Die Zielarchitektur soll Darstellung, Geschäftslogik und Datenzugriff trennen. Die Verarbeitung fachlicher Aufrufe soll in Controller, Services und Repositories gegliedert werden. Der Renderer soll die Oberfläche bereitstellen; privilegierte Operationen sollen über begrenzte Preload-Schnittstellen an den Main-Prozess vermittelt werden. Die Kommunikation soll über Interprozesskommunikation erfolgen, ohne hierfür einen lokalen HTTP-Server zu betreiben. Die Prozessaufteilung und die fachliche Schichtentrennung ergänzen sich, sind jedoch nicht gleichzusetzen.

Funktional soll ein zusammenhängender, sinnvoll nutzbarer Kernumfang der ursprünglichen Anwendung erhalten bleiben. Entscheidend ist die Abdeckung der unterschiedlichen Interaktionsformen: Oberflächeninteraktionen mit lokaler Datenverarbeitung, das Einlesen und Validieren von Dateien sowie Netzwerkanfragen und die Synchronisierung gemeinsamer Aufgaben. Dashboard, Kalender, Noten und Studiengruppen bilden den fachlichen Kontext dieser allgemeineren Anwendungsaufgaben. Die vollständige Übernahme jeder CRUD-Variante ist demgegenüber kein eigenständiges Ziel. Der Kernumfang muss für die betrachteten Interaktionsformen vollständige Verarbeitungspfade einschließlich Datenzuordnung, Validierung und Fehlerbehandlung bereitstellen.

Lokale Datenhaltung bildet die Grundlage der Offline-Kernfunktionen. Local-First verbindet lokale Nutzbarkeit mit weitergehenden Eigenschaften wie Kollaboration und Kontrolle über die eigenen Daten. [7] Die Arbeit greift daraus gezielt die lokale Verarbeitung und die ergänzende Synchronisierung im ausgewählten Gruppenfeature auf. Der Katalogabruf ergänzt diesen Umfang als Beispiel einer externen Datenübernahme.

Erweiterbarkeit bedeutet in diesem Zusammenhang, dass eine Person nach Einarbeitung in die dokumentierten Muster zusätzliche Funktionen mit vergleichbaren Anforderungen entsprechend umsetzen kann. Die vorhandenen Beispiele müssen deshalb sowohl den fachlichen Ablauf als auch die erforderlichen Validierungen, Berechtigungsprüfungen und Sicherheitsgrenzen nachvollziehbar zeigen. Das Fortführen dieser Muster soll zu einer konsistenten Anwendung führen, ohne für jede weitere Funktion einen neuen Architekturansatz entwickeln zu müssen.

Die Anwendung soll außerdem strukturell auf spätere Tests vorbereitet sein. Fachliche Services müssen unabhängig von Oberfläche und Transport prüfbar sein; Datenzugriffe und externe Dienste sollen

für diese Tests durch geeignete Testimplementierungen ersetzt werden können. Zusätzliche Testfälle und Funktionen innerhalb der betrachteten Interaktionsformen sollen sich ergänzen lassen, ohne zuvor die Schichtentrennung oder die grundlegenden Abhängigkeiten neu ordnen zu müssen. Damit wird die Vermeidung solcher nachgelagerten Strukturkorrekturen als Ziel formuliert; die tatsächliche Testabdeckung wird gesondert bewertet.

Unter einer sicheren beziehungsweise gehärteten Electron-Anwendung versteht diese Arbeit die Umsetzung der anwendbaren Electron Security Guidelines im festgelegten Prüfumfang. Dazu gehören insbesondere die Isolation des Renderers, begrenzte privilegierte Schnittstellen, die Prüfung von IPC-Absendern und Eingaben sowie Beschränkungen für nachgeladene Inhalte, Navigation und zusätzliche Fenster. [3] Der Begriff bezeichnet damit einen konkret prüfbaren Richtlinienrahmen und keinen Nachweis der Abwesenheit sämtlicher Schwachstellen.

4.4 Anforderungskatalog

Die Anforderungen verbinden die Fallauswahl, das Architekturziel und den festgelegten funktionalen sowie sicherheitsbezogenen Umfang. Die Kennungen A1–A9 werden für die Zuordnung der Migrationsschritte und die spätere Evaluation verwendet. Die nachfolgend genannten Bewertungsansätze legen fest, worauf ein Nachweis Bezug nehmen muss; konkrete Szenarien und Ergebnisse werden in Kapitel 6 dokumentiert.

4.4.1 A1: Lauffähige Desktop-Anwendung

Das Replit-Artefakt muss in eine lauffähige Electron-Anwendung überführt werden. Bewertet werden der Start einer gebauten Anwendung und die Ausführbarkeit der eingeschlossenen Kernfunktionen. Die untersuchte Version und die Laufzeitumgebung sind im Nachweis festzuhalten.

4.4.2 A2: Dreischichtenarchitektur

Darstellung, Geschäftslogik und Datenzugriff müssen getrennte Verantwortlichkeiten besitzen; fachliche Aufrufe sind entsprechend in Controller, Services und Repositories zu gliedern. Abhängigkeiten sind so zu gestalten, dass die Geschäftslogik unabhängig von Oberfläche und Transport mit austauschbaren Datenzugriffen geprüft werden kann. Der Nachweis erfolgt durch den Abgleich des Architekturmodells mit den tatsächlichen Modulen und einem beispielhaften fachlichen Ablauf. Die Electron-Prozesse sind dabei nicht mit den fachlichen Schichten gleichzusetzen; Geschäftslogik und Datenzugriff können beide im Main-Prozess liegen.

4.4.3 A3: Begrenzung des Refactoring-Aufwands

Bestandteile des Ausgangsartefakts sollen soweit möglich weiterverwendet werden. Notwendige Anpassungen und Ersetzungen sind nach Komponenten beziehungsweise Schichten zu dokumentieren und zu begründen. Diese Anforderung verlangt nachvollziehbare Eingriffe, jedoch keinen Nachweis eines global minimalen Entwicklungsaufwands.

4.4.4 A4: Lokale Kommunikation ohne HTTP-Wrapper

Die lokale Anwendungslogik darf nicht lediglich durch den Weiterbetrieb des ursprünglichen HTTP-Backends eingebettet werden. Der Nachweis beschreibt den tatsächlichen Aufrufweg über Preload und IPC und zeigt, dass für lokale fachliche Operationen kein HTTP-Server erforderlich ist. Externe HTTPS-Anbindungen bleiben davon unberührt.

4.4.5 A5: Lokale Datenhaltung und Offline-Kernfunktionen

Die für den Kernumfang ausgewählten lokalen Funktionen aus Abschnitt 4.2.1 und die lokale Identitätsverwaltung müssen ohne Netzwerkverbindung nutzbar sein. Änderungen müssen lokal gespeichert werden und einen Neustart der Anwendung überdauern. Für Funktionen mit Katalogdaten wird

dabei ein bereits lokal verfügbarer Datenbestand vorausgesetzt. Der Nachweis muss zwischen diesen Funktionen und ausdrücklich netzwerkabhängigen Operationen unterscheiden.

4.4.6 A6: Local-First-Kollaboration am Gruppenfeature „Aufgaben“

Aufgabenänderungen müssen im festgelegten Kollaborationsumfang lokal verarbeitet und bei verfügbarer Verbindung übermittelt werden können. Der Nachweis beschreibt lokale Speicherung und anschließende Synchronisierung als getrennte Vorgänge. Gruppenerstellung und Einladungsfunktionen gehören zum Untersuchungsumfang; ihre unmittelbar netzwerkabhängigen Operationen sind gesondert auszuweisen. Kommentare sind ausgeschlossen. Eine Mehrgeräte-Synchronisierung oder gemeinsame Echtzeitbearbeitung von Dokumenten wird nicht verlangt.

4.4.7 A7: Härtung gemäß Electron Security Guidelines

Die offizielle Sicherheitscheckliste muss vollständig auf Anwendbarkeit geprüft werden. Für anwendbare Empfehlungen sind Umsetzung, Beleg und Nachweisebene zu dokumentieren. Eingabevalidierung und Maßnahmen gegen fremden Code werden durch geeignete vorhandene Tests und ausgewählte negative Laufzeitfälle ergänzt. Eine fehlende Prüfung darf nicht als Erfüllung gewertet werden; begrenzt belegte Aussagen müssen ihre Nachweisgrenze nennen.

4.4.8 A8: Repräsentativer Kernfunktionsumfang und Erweiterbarkeit

Der Demonstrator muss einen zusammenhängenden Kernfunktionsumfang mit repräsentativen Abläufen für Oberflächeninteraktionen, lokale Persistenz, Dateiimporte und Netzwerkanbindungen bereitstellen. Gruppenerstellung, Einladungen und synchronisierte Aufgaben bleiben Bestandteil dieses Umfangs. Die vollständige Übernahme jeder CRUD-Variante wird nicht verlangt. Der Nachweis benennt die tatsächlich übernommenen Funktionen, vergleicht deren Verhalten mit dem Ausgangsartefakt und ordnet ihnen die verwendeten Architektur- und Sicherheitsmuster zu. Er muss nachvollziehbar machen, wie weitere Funktionen mit vergleichbaren Anforderungen auf diesen Mustern aufbauen können. Änderungen der Identitäts- und Sitzungsverwaltung sind zu begründen. Vorhandene Service-Unit-Tests ergänzen den Nachweis für die tatsächlich geprüfte Logik.

4.4.9 A9: Nachvollziehbarer Migrationsleitfaden

Der Leitfaden muss Voraussetzungen, Reihenfolge, Änderungen, Entscheidungen und Kontrollpunkte der durchgeführten Migration beschreiben. Seine Schritte müssen dem Demonstrator zugeordnet werden können. Projektspezifische Anpassungen und mögliche übertragbare Prinzipien sind kenntlich zu machen. Ein zusätzlicher unabhängiger Reproduktionsdurchlauf ist kein Bestandteil dieser Anforderung.

4.5 Abgrenzung des Untersuchungsumfangs

Untersucht werden die Migration und Härtung der Desktop-Anwendung anhand eines repräsentativen Kernumfangs. Dieser umfasst drei Interaktionsformen:

- **CRUD-Oberflächeninteraktionen:** Anlegen, Lesen, Bearbeiten und Löschen von Anwendungsdaten über die Oberfläche, einschließlich Eingabevalidierung, fachlicher Verarbeitung und lokaler Persistenz. Die Noten-, Termin- und Stundenplanverwaltung liefern hierfür die betrachteten Beispiele.
- **Dateiverarbeitung:** Einlesen, Parsen und Validieren lokaler CSV- und iCal-/ICS-Dateien sowie die Übernahme ihrer Inhalte in das interne Datenmodell. Betrachtet wird damit der vollständige Verarbeitungspfad eines Dateiimports.
- **Netzwerkinteraktionen:** Abruf und Verarbeitung externer Katalogdaten sowie die Anbindung der Gruppenfunktionen mit Authentifizierung, Gruppenerstellung, Einladungen und Synchronisie-

rung gemeinsamer Aufgaben. Unmittelbare Online-Anfragen und nachgelagerte Synchronisierung werden unterschieden.

Der Sicherheitsumfang bezieht sich auf die Desktop-Anwendung und ihre Schnittstellen. Eingabevalidierung, anwendungsinterne Zugriffskontrollen und die sichere Anbindung sowie Verarbeitung externer Daten bleiben Bestandteil der Untersuchung.

Nicht untersucht werden die interne Sicherheit und der sichere Betrieb der entfernten Dienste, die Synchronisierung oder die Bereitstellung von Nutzdaten übernehmen. Die Härtung ihrer Server, Datenbanken und sonstigen Infrastruktur liegt außerhalb des Umfangs. Die Nutzung solcher Dienste begründet somit keine Aussage über deren eigene Sicherheit.

Ebenfalls ausgeschlossen ist der Schutz der lokalen Datenbank innerhalb des Betriebssystems, insbesondere die Verschlüsselung der Datenbankdatei und ihre Absicherung gegen Zugriffe anderer lokaler Prozesse oder Betriebssystembenutzer. Die Datenzuordnung und Zugriffskontrolle innerhalb der Anwendung sowie die Validierung vor Datenbankoperationen bleiben davon unberührt.

Die Local-First-Demonstration bleibt auf gemeinsame Aufgaben und die zugehörige Gruppen- und Einladungsverwaltung begrenzt. Kommentare, weitere gemeinsame Gruppeninhalte, die Synchronisierung persönlicher Daten über mehrere Endgeräte, gemeinsame Echtzeitbearbeitung von Dokumenten und Bucket-Dateifunktionen sind ausgeschlossen.

Frontend-Unit-Tests, automatisierte End-to-End-Tests und ein umfassender Sicherheitsvergleich zwischen Web- und Desktop-Laufzeit sind ausgeschlossen. Diese Abgrenzungen beschränken den Geltungsbereich der Ergebnisse und gelten nicht als unerfüllte Anforderungen.

Kapitel 5

Entwurf, Migration und Migrationsleitfaden

Dieses Kapitel dokumentiert den durchgeführten Migrationsprozess und die dabei getroffenen Entscheidungen. Die Darstellung beginnt mit der Vorbereitung der lokalen Datenhaltung und folgt anschließend den weiteren Transformationsschritten. Auf dieser Grundlage werden die Handlungsempfehlungen des Migrationsleitfadens abgeleitet. Die Beschreibung der konkreten Umsetzung und die daraus gewonnenen Empfehlungen sind dabei zu unterscheiden.

5.1 Struktur des Migrationsleitfadens

Der Leitfaden ordnet die Migration in drei Phasen: die Überführung in eine lokal nutzbare Offline-Anwendung, die Ergänzung um Synchronisierung und weitere Online-Anbindungen sowie die konsolidierte Härtung. Vorausgesetzt wird die Analyse des Ausgangsartefakts aus Kapitel 4. Das dort erstellte Komponentenbild und der Anforderungskatalog legen fest, welche Funktionen übernommen und welche Verantwortlichkeiten verändert werden müssen.

Die Phasen unterscheiden Zielzustände der Entwicklung. Sie behaupten weder eine von Beginn an vollständig festgelegte Projektplanung noch eine vollständige Trennung aller Arbeitsschritte. Insbesondere beeinflussen die Sicherheitsanforderungen bereits den ersten Architekturentwurf. Jede Phase beschreibt deshalb neben ihrem funktionalen Ziel auch die relevanten Schnittstellen und einen Kontrollpunkt für den erreichten Stand.

Phase	Zielzustand	Kontrollpunkt
1	Lokale Kernfunktionen mit lokaler Datenhaltung und begrenzter IPC-Kommunikation.	Kernabläufe ohne Netzwerk ausführen; Änderungen nach Neustart erneut lesen.
2	Synchronisierte Aufgaben und gesonderte Online-Anbindungen.	Lokale Speicherung, spätere Übermittlung und Verhalten bei Netzwerkausfall unterscheiden.
3	Geprüfte Härtung im festgelegten Electron-Sicherheitsrahmen.	Anwendbarkeit, Umsetzung und Nachweis jeder Empfehlung dokumentieren.

Tabelle 5.1: Phasen und Kontrollpunkte des fallbezogenen Migrationsleitfadens.

5.2 Phase 1: Von der Webanwendung zur Offline-Kernanwendung

Die erste Phase überführt die persönlichen Kernfunktionen in eine lokale Desktop-Anwendung. Datenhaltung, Identitätsverwaltung, CRUD-Operationen und Dateiimporte werden innerhalb der Electron-Laufzeit ausgeführt. Synchronisierung und externe Katalogaktualisierung folgen in Phase 2. Für katalogabhängige Funktionen wird zunächst ein lokal bereitgestellter Datenbestand verwendet.

5.2.1 Initialisierung des Electron-Projekts und Übernahme der Datenbankmodule

Electron wurde im Sinne von A3 gewählt, um vorhandene Komponenten weiterzuverwenden. Das Backend des Ausgangsartefakts verwendete TypeScript und Node.js. [6] Da der Electron-Main-Prozess ebenfalls eine Node.js-Laufzeit bereitstellt, konnten fachliche Logik und Bibliotheken als Grundlage der lokalen Umsetzung dienen. [8]

Als Alternative wurde Tauri erwogen, dessen privilegierter Core-Prozess auf Rust basiert. [9] Eine Portierung des Backends in diesen Core hätte neben der Sprache auch Datentypen, Fehlerbehandlung und Bibliotheksanbindungen betroffen, insbesondere den Drizzle-Datenzugriff. Tauri erlaubt allerdings auch die Einbindung einer Node.js-Anwendung als Sidecar. [10] Dieser Weg hätte einen zusätzlichen auszuliefernden Prozess und dessen Kommunikation erfordert. Für Electron sprach die unmittelbare Ausführung der übernommenen Backend-Komponenten im Main-Prozess.

Das Zielprojekt wurde mit der `react-ts`-Vorlage von electron-vite initialisiert. [11] Damit blieb die im Replit-Frontend verwendete Kombination aus React, TypeScript und Vite erhalten. [6] Die betrachtete React-TypeScript-Anleitung von Electron Forge verwendet `webpack-typescript`; Forge bietet daneben auch Vite-Vorlagen an. [12], [13] Die Wahl von electron-vite beruhte auf der erwarteten Begrenzung der Anpassungen durch kompatible Technologien. Ein geringerer Migrationsaufwand gegenüber Tauri oder Electron Forge wurde nicht vergleichend evaluiert.

Anschließend wurden die Drizzle-Schemadefinitionen für Benutzer, Stundenpläne, Studiengänge, Gruppen, Noten, Termine und das Aktivitätsprotokoll samt Exportmodul nach `src/main/db/schema` übernommen. Die angepasste `drizzle.config.ts` liegt im Projektstamm. Für die erste Entwicklung ersetzte eine lokale SQLite-Datei die PostgreSQL-Verbindung: Der Dialekt wechselte zu `sqlite`, und `DATABASE_URL` entfiel für die lokale Datenhaltung. [6], [14]

Mit dem Umgebungswechsel übernahm die Desktop-Anwendung im Main-Prozess das Anlegen, Öffnen und Migrieren der Benutzerdatenbank. Die Datei wird zur Laufzeit unter dem von Electron bereitgestellten Pfad `userData` angelegt. [15] Dieser liegt außerhalb des Projekt- und Installationsverzeichnisses, trennt veränderliche Daten von Programmdateien und vermeidet Schreibzugriffe auf das Installationsverzeichnis. Ein Update kann so Programmdateien ersetzen, ohne die Benutzerdatenbank auszutauschen. Ausgeliefert werden die Migrationen zur Einrichtung und Aktualisierung ihres Schemas. [14]

5.2.2 Anpassung der Tabellen und Datenbankabhängigkeiten an SQLite

Die Übernahme der Dateien genügte noch nicht, da die bestehenden Definitionen PostgreSQL-spezifische Datentypen und Tabellenkonstruktoren verwendeten. Diese wurden durch die entsprechenden SQLite-Konstrukte ersetzt. Die fachlichen Tabellen- und Spaltennamen sowie die vorhandenen Pflichtfelder und Standardwerte wurden dabei möglichst beibehalten. Dieser Ansatz begrenzte die Änderungen am Datenmodell und entsprach dem in Kapitel 4 festgelegten Ziel, vorhandene Komponenten mit möglichst geringem Refactoring-Aufwand weiterzuverwenden.

Die Anpassung lässt sich am Stundenplanschema nachvollziehen. Quellcode 5.1 zeigt die ursprüngliche Definition der Tabelle `timetable_entries`; Quellcode 5.2 zeigt ihre Überführung in das SQLite-Schema des Demonstrators. Beide Ausschnitte lassen die übrigen Tabellen und die abgeleiteten Validierungsschemata weg; die Zeilenumbrüche wurden für die Darstellung angepasst. [6], [14]

```
import {
  pgTable, serial, text, integer, boolean, pgEnum
} from "drizzle-orm/pg-core";

export const weekDayEnum = pgEnum("week_day", [
  "monday", "tuesday", "wednesday", "thursday", "friday"
]);
export const timeSlotEnum = pgEnum("time_slot", [
  "slot1", "slot2", "slot3", "slot4", "slot5", "slot6"
]);

export const timetableEntriesTable = pgTable("timetable_entries", {
  id: serial("id").primaryKey(),
  courseName: text("course_name").notNull(),
  weekDay: weekDayEnum("week_day").notNull(),
  timeSlot: timeSlotEnum("time_slot").notNull(),
  room: text("room"),
  professor: text("professor"),
  isCustom: boolean("is_custom").notNull().default(true),
  color: text("color"),
  userId: integer("user_id").notNull(),
});
```

Quellcode 5.1: Stundenplaneinträge im PostgreSQL-Schema des Ausgangsartefakts (Ausschnitt).

```
import { sqliteTable, integer, text } from 'drizzle-orm/sqlite-core'

export const weekDays = [
  'monday', 'tuesday', 'wednesday', 'thursday', 'friday'
] as const
export type WeekDay = (typeof weekDays)[number]

export const timeSlots = [
  'slot1', 'slot2', 'slot3', 'slot4', 'slot5', 'slot6'
] as const
export type TimeSlot = (typeof timeSlots)[number]

export const timetableEntriesTable = sqliteTable('timetable_entries', {
  id: integer('id').primaryKey({ autoIncrement: true }),
  courseName: text('course_name').notNull(),
  weekDay: text('week_day', { enum: weekDays }).notNull(),
  timeSlot: text('time_slot', { enum: timeSlots }).notNull(),
  room: text('room'),
  professor: text('professor'),
  isCustom: integer('is_custom', { mode: 'boolean' })
    .notNull().default(true),
  color: text('color'),
  userId: integer('user_id').notNull()
})
```

Quellcode 5.2: Stundenplaneinträge im SQLite-Schema des Demonstrators (Ausschnitt).

An die Stelle von `pgTable` trat `sqliteTable`. Der automatisch erzeugte Primärschlüssel wurde von `serial` auf einen Integer-Primärschlüssel mit `autoIncrement` umgestellt. Das boolesche Feld `isCustom` wird in SQLite als Integer gespeichert; Drizzles Modus `boolean` erhält dabei die Verwendung boolescher Werte im TypeScript-Code. [16]

Die PostgreSQL-Enums für Wochentage und Zeitblöcke wurden durch konstante Wertelisten und daraus abgeleitete TypeScript-Union-Typen ersetzt. Die Spalten verwenden diese Listen über die `enum`-Option des Text-Konstruktors. Diese Option unterstützt die Typableitung, erzwingt jedoch zur

Laufzeit keinen zulässigen Wertebereich in der Datenbank. [16] Die erhaltene Typisierung ist daher nicht mit der bisherigen Datenbankprüfung gleichzusetzen; eine Prüfung eingehender Werte bleibt erforderlich. Die aus den Tabellen abgeleiteten Insert- und Select-Typen sowie die Erzeugung der Zod-Schemata konnten weiterhin nach dem vorhandenen Muster verwendet werden.

Für die benötigten Pakete wurde zunächst die Abhängigkeitskonfiguration des Ausgangsartefakts herangezogen. Die angegebenen Versionsbereiche für `drizzle-orm` (`^0.45.2`), `drizzle-kit` (`^0.31.10`) und `drizzle-zod` (`^0.8.3`) wurden übernommen. Der PostgreSQL-Treiber `pg` wurde für die lokale Datenhaltung durch `@libsql/client` ersetzt; im Demonstrator ist hierfür der Versionsbereich `^0.18.0` angegeben. [6], [14] Damit blieb Drizzle als Abstraktion für Schema und Datenzugriff erhalten, während der verwendete Treiber wechselte.

Die Verantwortlichkeiten der Pakete sind dabei zu unterscheiden: Der libSQL-Client öffnet die lokale Datenbankdatei, Drizzle ORM führt die Datenbankoperationen aus und Drizzle Kit erzeugt die SQL-Migrationsdateien. Das Anwenden dieser Dateien beim Anwendungsstart übernimmt der Migrator aus `drizzle-orm/libsql/migrator`. Die automatische Anwendung vorbereiteter Migrationen ist somit Bestandteil der eingerichteten Drizzle-Initialisierung und keine alleinige Eigenschaft des Datenbanktreibers. [14]

5.2.3 Versionierte Schemamigrationen und Datenbankinitialisierung

Nach der Anpassung der Tabellen wurden zwei Wege zur Einrichtung des Datenbankschemas unterschieden. Mit `drizzle-kit push` können die Schemadefinitionen unmittelbar mit einer angebundenen Datenbank abgeglichen und die Änderungen ohne gesondert gespeicherte SQL-Migrationsdateien angewandt werden. [17] Alternativ erzeugt `drizzle-kit generate` SQL-Dateien anhand der aktuellen Definitionen und der bisherigen Schema-Snapshots. Diese Dateien können anschließend durch den Drizzle-Migrator angewandt werden. [18]

Für den Demonstrator wurde der zweite Weg gewählt. Die Migrationsdateien liegen im Verzeichnis `drizzle` und können gemeinsam mit dem Anwendungscode versioniert und ausgeliefert werden. Beim Start verarbeitet der Migrator die noch nicht angewandten Migrationen. Damit steht ein nachvollziehbarer Ablauf für spätere Schemaänderungen bereit, ohne die vorhandene Benutzerdatenbank bei jedem Update neu anzulegen. Die Verträglichkeit einer konkreten Schemaänderung mit bestehenden Daten muss dennoch anhand der jeweiligen Migration geprüft werden.

Die resultierende Konfiguration beschreibt deshalb nur den SQLite-Dialekt, den Einstiegspunkt der Schemadefinitionen und das Ausgabeverzeichnis der Migrationen. Ein Verbindungsparameter zur Entwicklungsdatenbank ist für die Generierung nicht mehr erforderlich. [18] Der Laufzeitpfad der Benutzerdatenbank wird separat im Main-Prozess festgelegt. Quellcode 5.3 zeigt diese Konfiguration des Demonstrators. [14]

```ts
import { defineConfig } from 'drizzle-kit'

export default defineConfig({
  schema: './src/main/db/schema/index.ts',
  out: './drizzle',
  dialect: 'sqlite'
})
```

Quellcode 5.3: Drizzle-Kit-Konfiguration zur Erzeugung der SQLite-Migrationen.

In `package.json` wurden `db:generate` für `drizzle-kit generate` und `db:check` für `drizzle-kit check` eingerichtet. Zur Laufzeit verwendet die Anwendung die erzeugten Dateien über den ORM-Migrator. [14]

Die Initialisierung wurde in den Startablauf des Main-Prozesses eingebunden. Innerhalb des mit `app.whenReady()` begonnenen Ablaufs in `src/main/index.ts` wird `initDatabase()` abgewartet, bevor die lokale Datenbank an Repositories und Dienste übergeben und das Anwendungsfenster geöffnet wird. Scheitert die Vorbereitung, zeigt die Anwendung eine Fehlermeldung an und beendet den Start. Die Oberfläche wird dadurch nicht mit einer unvollständig vorbereiteten Datenbank geöffnet. [14]

In `src/main/db/index.ts` wird der Speicherort aus `app.getPath('userData')` und dem Unterverzeichnis `data` gebildet. Die paketierte Anwendung verwendet `studyplan.db`, die Entwicklung `studyplan-dev.db`. Unter Windows ergibt sich beispielsweise `%APPDATA%\studyplan\data\studyplan.db` innerhalb von `AppData\Roaming`. Der Unterordner wird vor dem Öffnen der Datei angelegt. [14]

Danach öffnet der libSQL-Client die Datei über eine lokale `file:`-URL. Die Initialisierung setzt ausdrücklich `PRAGMA foreign_keys = ON` und `PRAGMA journal_mode = WAL`. Die erste Einstellung aktiviert die Prüfung der im Schema tatsächlich definierten Fremdschlüsselbedingungen; sie ergänzt selbst keine solchen Bedingungen. Die zweite wählt den Write-Ahead-Logging-Modus für die Datenbank. [19] Anschließend werden die ausstehenden Migrationen angewandt und die Verbindung mit einer einfachen Abfrage geprüft. Die konkrete Implementierung legt außerdem die SQLite-kompatible Migrationstabelle `__drizzle_migrations` vor dem Aufruf des Migrators an. [14]

Damit die Migrationen auch in der installierten Anwendung verfügbar sind, wurde das Verzeichnis `drizzle` in `electron-builder.yml` unter `extraResources` aufgenommen. Quellcode 5.4 zeigt diesen Eintrag. In der Entwicklung liest die Anwendung die Dateien aus dem Projektverzeichnis; im paketierten Zustand verwendet sie `process.resourcesPath` und den dort enthaltenen Unterordner `drizzle`. [14]

```yaml
extraResources:
  - from: drizzle
    to: drizzle
    filter:
      - '**/*'
```

Quellcode 5.4: Auslieferung der Migrationsdateien über zusätzliche Paketressourcen (Ausschnitt).

Für Funktionen mit Referenzdaten wurden außerdem Studiengänge, Lehrveranstaltungen und Stundenplanvorlagen über die Datenmigrationen `0001_seed-study-programs.sql` und `0002_seed-timetable-presets.sql` bereitgestellt. Sie werden beim Einrichten der Datenbank lokal eingefügt und stehen damit auch vor dem in Phase 2 ergänzten Katalogabruf zur Verfügung. Dies ist für die Offline-Nutzung entscheidend: Ein lokal ausführbarer Handler kann nur auf den tatsächlich vorhandenen Datenbestand zugreifen. [14]

5.2.4 Übernahme des API-Vertrags und Anpassung der Codegenerierung

Die Schnittstellenmigration ging vom OpenAPI-Vertrag unter `lib/api-spec/openapi.yaml` aus, der nach `api/openapi.yaml` übernommen wurde. Seine Operationskennungen, Parameter und Datenschemata dienten bereits im Web-Prototyp zur Erzeugung von Aufrufhilfen und Zod-Schemata. [6]

Im Ausgangsartefakt verwendete Orval dafür `orval.config.ts`. Im Desktop-Projekt übernehmen eigene TypeScript-Skripte unter `scripts/openapi-ipc` die Generierung. Ihre Konfiguration `api/ipc-generator.config.ts` legt den Eingabevertrag und die Ausgabepfade fest. Das Prinzip der

gemeinsamen Vertragsgrundlage bleibt damit erhalten, während die Umsetzung auf IPC ausgerichtet wird. [20]

Der Generator liest den Vertrag mit `yaml` und erzeugt TypeScript-Typen, Zod-Laufzeitschemata, die Zuordnungstabelle `operationContracts` sowie einen transportunabhängigen API-Client unter `src/shared/api/generated`. React-Query-Hooks liegen unter `src/renderer/src/api/generated`. Der Main-Prozess verwendet die gemeinsamen Schemata zur Validierung. [20]

`api:generate` erzeugt die Module; `api:check` prüft ihre Übereinstimmung mit der Generatorausgabe. Der YAML-Vertrag wird während der Entwicklung eingelesen. Zur Laufzeit werden die erzeugten Module verwendet. Die fachliche Implementierung und ihre Handler-Registrierung bleiben gesonderte Aufgaben. [20]

Während der Oberflächenintegration wurde `generator-lib.ts` im Commit `f03ab7f` um React-Query-Hooks und das Exportmodul `src/renderer/src/api/index.ts` erweitert. Die Aufrufformen orientieren sich am Web-Client: Queries erhalten Pfad- und Query-Parameter, Mutationen ein Variablenobjekt mit `data` für den bisherigen Request-Body. Auch Hilfsfunktionen für Query Keys werden erzeugt. Diese Orientierung ermöglicht die Weiterverwendung vorhandener Aufrufmuster. [20], [21]

Zunächst stellte Preload den vollständigen API-Client bereit. Im Authentifizierungscommit `1c1988d` wurde seine Erzeugung in den Renderer verlagert: Die Hooks verwenden dort `createApiClient` mit `window.apiTransport.invoke`. Preload vermittelt den Transport; der Client verarbeitet Ergebnisse und rekonstruiert Fehler als `ApiClientError`. [22]

5.2.5 IPC-Transport, Preload-Schnittstelle und Dispatcher

Der API-Client ersetzt HTTP-Methode, URL und `customFetch` durch eine Funktion vom Typ `ApiTransport`. Diese übermittelt `operationId` und typisierte Eingabedaten über Preload. Die übernommenen Gruppen `path`, `query` und `body` bezeichnen nun Felder des IPC-Eingabeobjekts. Die Schnittstelle folgt dem Prinzip eines Remote Procedure Call (RPC): Der Renderer ruft eine benannte Operation im Main-Prozess auf und erhält deren Ergebnis. Electron-IPC bildet dafür den lokalen Transport zwischen den Prozessen. [6], [20]

In `src/preload/index.ts` sendet `apiTransport.invoke` die Operationskennung, Eingaben und eine `requestId` über `ipcRenderer.invoke` auf dem festen Kanal `api:invoke`. `contextBridge.exposeInMainWorld` stellt dem Renderer `window.apiTransport` bereit; `ipcMain.handle` nimmt den Aufruf im Main-Prozess entgegen. Dies entspricht dem von Electron dokumentierten Aufrufmuster mit Rückgabewert. [20], [23]

`src/preload/index.d.ts` erweitert `Window` für die Typprüfung; das Laufzeitobjekt stellt Preload bereit. Der Renderer erhält die begrenzte Transportmethode statt frei wählbarer IPC-Kanäle oder des vollständigen `ipcRenderer`-Objekts. Diese Begrenzung entspricht der Electron-Empfehlung für die Bridge. [24]

Der Dispatcher in `src/main/ipc/api-dispatcher.ts` prüft Absender und Anfrageform und ordnet die `operationId` einem Vertrag sowie einem registrierten Handler zu. Das generierte Zod-Schema validiert die Eingaben. Unbekannte Operationen und ungültige Daten werden abgewiesen. [20]

Nach der Handler-Ausführung validiert der Dispatcher auch Rückgabewerte und ausdrücklich deklarierte Fehlerantworten. Ein erfolgreiches `ApiResult` enthält `ok: true`, Daten und Anfragekennung; ein Fehler enthält `ok: false` und ein Fehlerobjekt. Der API-Client entnimmt die Daten beziehungsweise erzeugt `ApiClientError`. [20]

Bei einer bekannten, aber nicht registrierten Operation liefert der Dispatcher `NOT_IMPLEMENTED`. Generierte Hooks allein stellen somit keine ausführbare Funktion bereit. Fachlogik, Sitzungszuordnung

und Berechtigungen werden in Handlern und Services umgesetzt; die konsolidierte Härtungsprüfung folgt in Phase 3. [20]

5.2.6 Begründung und Grenzen der vertragsbasierten Generierung

Alternativ hätten die von Orval erzeugten Hooks, Typen und Zod-Schemata manuell angepasst werden können. Der eigene Generator wäre entfallen, Änderungen hätten jedoch in mehreren abhängigen Modulen abgestimmt werden müssen. Für den erweiterbaren Demonstrator wurde deshalb die zentrale Vertragsgrundlage beibehalten und ihre Übersetzung nach IPC automatisiert.

Für Erweiterungen wird der Vertrag an einer Stelle ergänzt und die Änderung durch erneute Generierung in Typen, Client, Hooks und Laufzeitschemata übertragen. Anschließend werden Handler, Fachlogik und Oberflächenabläufe implementiert. Dieses Vorgehen unterstützt die Wiederverwendung nach A3 und das Erweiterungsmuster nach A8 und A9. Der anfängliche Entwicklungs- und spätere Pflegeaufwand des Generators bleibt zu berücksichtigen; eine quantitative Aufwandsersparnis wurde nicht nachgewiesen. [20]

Die gemeinsame Vertragsbindung definiert die Schnittstelle zwischen Renderer und Main-Prozess. Die fachliche Implementierung bleibt hinter dieser Grenze im Main-Prozess. Damit können Schnittstellenverarbeitung, Fachlogik und Datenzugriff getrennt geprüft werden, wie A2 verlangt.

Abbildung 5.1 stellt den ursprünglichen Aufrufweg demjenigen der lokalen Electron-Kernanwendung gegenüber. In beiden Fällen bildet ein gemeinsamer Vertrag die Grundlage für Client und Validierung. Die wesentliche Änderung liegt in der Umsetzung dieses Vertrags zur Laufzeit: HTTP-Aufrufe an einen Express-Server werden durch begrenzte Aufrufe über Preload und IPC ersetzt. Die eingezeichnete Trennung in Handler, Services und Repositories zeigt die resultierende Desktop-Architektur; ihre fachliche Ausarbeitung ist ein weiterer Migrationsschritt. [6], [20]

Die Umsetzung der gemeinsamen Vertragsgrundlage im Web-Prototyp und in der Electron-Kernanwendung wird in Tabelle 5.2 gegenübergestellt.

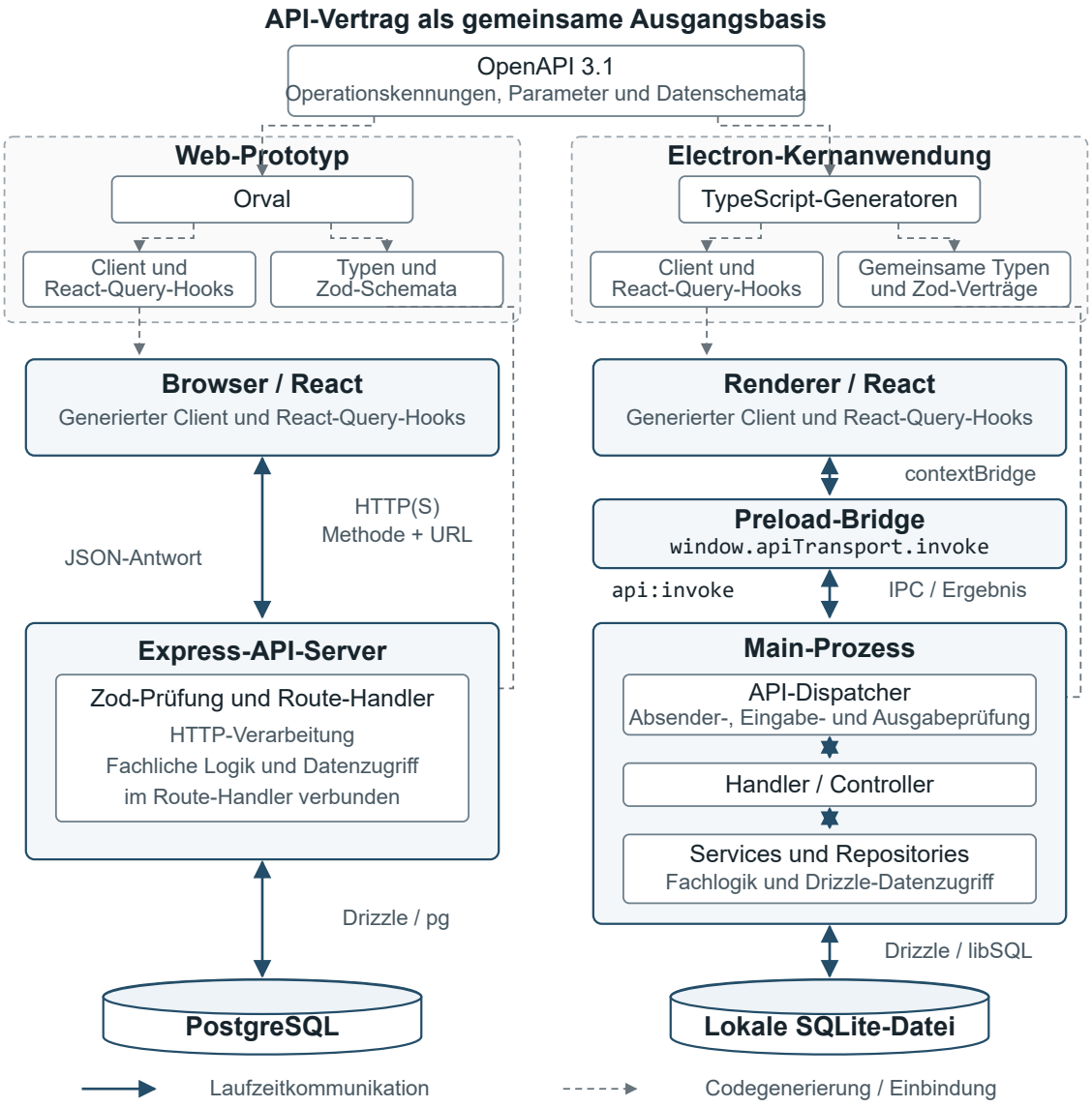


Abbildung 5.1: Vertragsbasierte Codegenerierung und Aufrufwege im Web-Prototyp und in der lokalen Electron-Kernanwendung. Gestrichelte Pfeile zeigen Generierung oder Einbindung, durchgezogene Pfeile Laufzeitkommunikation. Spätere Synchronisierungs- und Online-Pfade sind nicht dargestellt.

Aspekt	Web-Prototyp	Electron-Kernanwendung
Vertragsgrundlage	OpenAPI mit Operationskennungen und Datenschemata.	Bestehende Definitionen übernommen und gezielt ergänzt.
Generierung	Orval für React Query und Zod.	Projektspezifische TypeScript-Generatoren für Client, Hooks und gemeinsame Zod-Verträge.
Aufrufweg	HTTP-Methode und URL über `customFetch`.	`operationId` und Eingabeobjekt über Preload und den Kanal `api:invoke`.
Parameter	Pfad, Query-String und HTTP-Body.	Strukturierte Eingabe mit `path`, `query` und `body`.
Ergebnis	HTTP-Antwort beziehungsweise HTTP-Fehler.	Geprüftes `ApiResult`; Fehler werden als `ApiClientError` weitergegeben.
Datumswerte	Strings im generierten Client; Datumsprüfung in Zod-Schemata.	`Date`-Werte und `z.date()` in den gemeinsamen Modulen.
Zugriffsgrenze	Serverseitige HTTP-Verarbeitung und Sitzungszuordnung.	Begrenzte Bridge, Absender- und Schemaprüfung sowie fachliche Sitzungsprüfung.

Tabelle 5.2: Wiederverwendete Vertragsgrundlage und Änderungen an ihrer Umsetzung.

OpenAPI beschreibt standardmäßig HTTP-APIs. [25] Die IPC-Nutzung übersetzt ausgewählte Vertragsbestandteile projektspezifisch: Methoden und Pfade bleiben Metadaten, die Weiterleitung verwendet die Operationskennung. Der Generator unterstützt die benötigten Schemakonstrukte und lokale Referenzen, weist jedoch beispielsweise rekursive Komponentenschemata zurück. Er bildet damit den verwendeten Vertragsumfang ab, ohne vollständige OpenAPI-Kompatibilität zu beanspruchen. [20]

5.2.7 Übernahme der React-Oberfläche in den Renderer

Die React-Oberfläche aus `artifacts/studienplaner/` wurde im Commit `f4ea658` nach `src/renderer/src/` übernommen. Dazu gehören Layout, UI-Komponenten, Hilfsmodule unter `hooks/` und `lib/` sowie die Seiten für Dashboard, Kalender, Noten und Stundenplan. [21]

`App.tsx` und `main.tsx` behielten die Einbindung von React Query über `QueryClientProvider` sowie die Anbieter für Tooltips und Benachrichtigungen bei. Die Darstellungsstruktur konnte weitgehend weiterverwendet werden; anzupassen waren insbesondere Modulauflösung, Datenzugriffe und Navigation. [6], [21]

Die übernommene `index.css` enthält Theme-Variablen und die Einbindung von Tailwind CSS, `tw-animate-css` und dem Typografie-Plugin. Die Vorlagendateien `assets/main.css` und `assets/base.css` wurden samt Beispielkomponenten und Grafiken entfernt. Der zunächst ebenfalls kopierte Google-Fonts-Import wurde im Commit `36ff5f9` entfernt; die vorhandenen Schriftstapel enthalten lokale Systemschrift-Fallbacks. Damit wird für die Darstellung kein externer Schriftabruf vorausgesetzt. [21]

Mitkopierte Paket- und Konfigurationsdateien wurden nach dem Abgleich mit der Electron-Struktur im Commit `8b9bd66` entfernt: `package.json`, `vite.config.ts`, `tsconfig.json`, `components.json` und die zusätzliche `index.html`. Der Renderer verwendet `src/renderer/index.html` und die Paket- und Build-Konfiguration des Zielprojekts. [21]

Die Replit-Plugins, `PORT`, `BASE_PATH` sowie Webserver- und Ausgabeeinstellungen der ursprünglichen Vite-Konfiguration wurden nicht übernommen. [6] electron-vite verwaltet Main, Preload und Renderer in einer gemeinsamen Konfiguration; für den Renderer verwendet es standardmäßig `src/renderer` und die dortige `index.html`. [26]

Im Commit `ceccf8f` wurde der Alias `@` in `electron.vite.config.ts` unter `renderer.resolve.alias` auf `src/renderer/src` gesetzt. `tsconfig.web.json` erhielt die entsprechende Zuordnung für `@/*`; `@renderer` blieb erhalten. So konnten Imports wie `@/components/ui/badge` weiterverwendet werden. [21] Vite benötigt die Zuordnung für den Build, TypeScript für die Modul- und Typprüfung. TypeScript-`paths` allein schreibt die ausgegebenen Importpfade nicht um. [27]

Für die Styles wurde `@tailwindcss/vite` mit `tailwindcss()` neben `react()` registriert, entsprechend dem dokumentierten Tailwind-Integrationsweg. [28] Diese Anpassung und die Vite-Typreferenz in `src/renderer/src/env.d.ts` gehören zur Oberflächenintegration im Commit `f03ab7f`; dessen Generatoränderungen wurden bereits in Abschnitt 5.2.4 behandelt. [21]

Die Paketdefinition wurde anhand des Ausgangsartefakts um Radix UI, `clsx`, `class-variance-authority`, `tailwind-merge`, React Hook Form samt Resolver, React Query, `date-fns`, Lucide und Wouter ergänzt. Die shadcn/ui-Komponenten wurden als Quelldateien übernommen. Die Abhängigkeitsauflösung entstand im Zielprojekt neu: Das Ausgangsartefakt verwendet `pnpm-lock.yaml`, das Electron-Projekt `package-lock.json`. [6], [21]

5.2.8 Anbindung der Seiten, Datumswerte und Navigation

Die bisherigen Imports aus `@workspace/api-client-react` wurden in den Seiten und im Layout durch Imports aus `@/api` ersetzt. Dieses Modul exportiert die für IPC erzeugten Hooks und zugehörigen Typen. Die Generatoranpassungen aus Abschnitt 5.2.4 ermöglichten, vorhandene Hook-Namen und typische Aufrufmuster beizubehalten. Beispielsweise blieb die Verwendung von `useListEvents(monthParams)` beziehungsweise einer Mutation mit `{ data: ... }` erhalten. Die Änderung des Imports allein genügte allerdings dort nicht, wo sich die Datendarstellung an der Schnittstelle änderte. [21]

Im Commit `ce275e4` wurde beispielsweise in `pages/kalender.tsx` die Verarbeitung von Datumsstrings an die generierten `Date`-Typen angepasst. `eventsForDay` verwendet `new Date(e.startDate)` anstelle von `parseISO`; für das Enddatum gilt dieselbe Anpassung. Formulareingaben werden in `Date`-Werte umgewandelt und Abfragegrenzen als solche übergeben. Vorhandene Werte können direkt formatiert oder über `getTime()` sortiert werden. Das Dashboard wurde entsprechend angepasst. [21]

Ungenutzte Imports, Variablen und Zustandseinträge wurden ebenfalls entfernt. Die geerbte Electron-Toolkit-Konfiguration aktiviert `noUnusedLocals` und `noUnusedParameters`; das Ausgangsartefakt setzte `noUnusedLocals` auf `false`. [6], [21] Die Bereinigung war daher auch für die Typprüfung erforderlich, da TypeScript ungenutzte lokale Deklarationen bei aktivierter Prüfung als Fehler meldet. [29]

Die direkten HTTP-Dateiimporte der kopierten Kalender- und Notenseiten wurden im nachfolgend beschriebenen Importschritt durch lokale Operationen ersetzt. [6], [21]

Für die Navigation wurde das bereits verwendete Paket `wouter` beibehalten. Im Web-Prototyp erhielt dessen Router den aus `import.meta.env.BASE_URL` abgeleiteten Basispfad. Im Authentifizierungscommit `1c1988d` wurde stattdessen `useHashLocation` aus `wouter/use-hash-location` eingebunden und mit `<WouterRouter hook={useHashLocation}>` aktiviert. [21] Der Hook verwendet den Teil der Adresse hinter `#` als Anwendungsroute. [30] Eine Seite wie `/kalender` kann dadurch innerhalb der geladenen Renderer-Ressource als `#/kalender` adressiert werden. Der Pfad der lokalen HTML-Datei bleibt bestehen, während Wouter die Seitenauswahl übernimmt. Die vorhandenen React-Routen und Navigationskomponenten konnten auf dieser Grundlage weiterverwendet werden. [21]

5.2.9 Lokale Authentifizierung als erster vollständiger Verarbeitungspfad

Nach der Übernahme der Oberfläche begrenzte der vorhandene Anmeldeablauf zunächst den Zugang zu den fachlichen Seiten. Das Layout ruft `useGetMe` auf und setzt einen zugeordneten Benutzer voraus. Für die weitere Entwicklung wurde die Anmeldung deshalb beibehalten und als erste Handlergruppe implementiert. Damit konnte der bereits vorbereitete Aufrufweg zwischen Oberfläche, Preload, Dispatcher und lokaler Datenhaltung an einem zusammenhängenden Ablauf genutzt werden. Dieser Schritt ist im Commit `1c1988d` dokumentiert. [22]

`auth.handlers.ts` ordnet `register`, `login`, `logout` und `getMe` den Service-Aufrufen zu und übersetzt deklarierte Fehler. `AuthService` übernimmt Registrierung, Passwortprüfung und Benutzerzuordnung; `UsersRepository` führt die Drizzle-Abfragen aus. Nach der Datenbankinitialisierung werden die Module erzeugt und die Handler registriert. Damit ist die in A2 geforderte Trennung erstmals für eine vollständige Operationsgruppe umgesetzt. [22]

Für das Hashen und Vergleichen der Passwörter wurde das bereits im Web-Backend verwendete Paket `bcryptjs` mit dem dort angegebenen Versionsbereich `^3.0.3` übernommen. Passwort-Hashes bleiben im Main-Prozess und in der lokalen Benutzertabelle; die Oberfläche erhält nur die vorgesehenen Benutzerdaten. An die Stelle der HTTP-Sitzung tritt `SessionService`, der die aktuelle Benutzerkennung im Main-Prozess hält. Diese lokale Sitzung liegt im Arbeitsspeicher und endet beim Beenden der Anwendung; die Benutzerdatensätze bleiben dagegen in der SQLite-Datenbank gespeichert. Die lokale Anmeldung ist von der später ergänzten Authentifizierung gegenüber entfernten Kollaborationsdiensten zu unterscheiden. [6], [22]

Bei erfolgreicher Anmeldung oder Registrierung leert der Renderer den bisherigen React-Query-Cache, speichert die Benutzerinformation unter dem Query Key von `getMe` und navigiert zur Anwendung. `UNAUTHENTICATED` führt zur Anmeldung und wird von der Query-Wiederholungslogik berücksichtigt. Andere Ladefehler erhalten eine Anzeige mit erneuter Abfragemöglichkeit. Die Weiterleitung bei bestehender Anmeldung erfolgt über einen React-Effekt. [22]

5.2.10 Integration der CRUD-Funktionen am Beispiel der Notenverwaltung

Nach der Oberflächenintegration und der lokalen Authentifizierung wurden die weiteren fachlichen Funktionen mit dem vorbereiteten Aufrufweg verbunden. Die Migration der CRUD-Operationen wird exemplarisch an der Notenverwaltung beschrieben. Anlegen, Lesen, Bearbeiten und Löschen folgen in den persönlichen Anwendungsbereichen einer wiederkehrenden Grundstruktur; eine separate Beschreibung jeder Variante würde überwiegend dieselben Transformationsschritte wiederholen. Die Notenverwaltung wurde im Commit `f97642b` integriert, zunächst ohne CSV-Import. Als fachliche Vorlage dienten die ursprünglichen Routen aus `artifacts/api-server/src/routes/grades.ts`. [6], [31]

Die Übertragung bestand darin, die innerhalb der Route verbundenen Aufgaben nach ihren Verantwortlichkeiten aufzuteilen. HTTP-spezifische Zugriffe auf `req` und `res` entfielen. Die Eingabeprüfung wurde durch die bereits eingerichtete Vertragsvalidierung des Dispatchers übernommen. Die Verarbeitung der Operationen und ihrer Fehler wurde den Handlern zugeordnet, die fachlichen Regeln dem Service und die Drizzle-Abfragen dem Repository. Tabelle 5.3 zeigt diese Zuordnung am untersuchten Beispiel. [31]

Schicht	Zielmodul	Aufgabe im Notenbeispiel
Schnittstelle	`grades.handlers.ts`	Operationszuordnung, Übernahme der Sitzungskennung und Übersetzung fachlicher Fehler.
Fachlogik	`GradesService`	Anlegen und Ändern fachlicher Werte, Notenstatistik und Veranlassung von Aktivitätseinträgen.
Datenzugriff	`GradesRepository`	Lesen und Schreiben der Notendatensätze über Drizzle mit benutzerbezogenen Auswahlbedingungen.

Tabelle 5.3: Aufteilung der ursprünglichen Notenrouten auf Handler, Service und Repository.

Die Module wurden in `src/main/index.ts` in den Startablauf eingebunden. Nach der Datenbankinitialisierung werden `GradesRepository` und `ActivityLogRepository` mit der Datenbankverbindung erzeugt. Beide werden an `GradesService` übergeben. `createGradesHandlers` verbindet anschließend die Service-Aufrufe mit den vorgesehenen Operationskennungen und der bereits vorhandenen `SessionService`-Instanz. Die resultierenden Handler werden in die gemeinsame Registry aufgenommen, die der IPC-Dispatcher verwendet. Die fachliche Implementierung wird damit an die zuvor erzeugten Schnittstellen angeschlossen. [31]

Die Benutzerkennung stammt bei diesen Operationen aus der lokalen Sitzung im Main-Prozess. Der gemeinsame Wrapper `authenticated` stellt sie dem Handler bereit. Im Repository werden Änderungen und Löschungen anhand der Datensatzkennung und der Benutzerkennung eingeschränkt. Findet eine solche Operation keinen passenden Datensatz, meldet der Service einen fachlichen Fehler, den der Handler in `NOT_FOUND` überführt. Damit bleibt die persönliche Datenzuordnung Bestandteil des vollständigen Verarbeitungspfads. [31]

Die vorhandenen fachlichen Regeln wurden als Ausgangspunkt beibehalten. Dazu gehören optionale Werte, die Erfassung von Aktivitäten, der ECTS-gewichtete Notendurchschnitt und die Auswertung nach Semester. Anpassungen ergaben sich aus der lokalen Datenrepräsentation: Note und ECTS werden im SQLite-Schema als numerische Werte verarbeitet, sodass die bisher für PostgreSQL verwendete Umwandlung in Strings beziehungsweise deren Rückkonvertierung entfallen konnte. Die Migration übernahm somit die fachlichen Berechnungen und gliederte ihre Ausführung neu. [6], [31]

Für die integrierten CRUD-Funktionen ergibt sich der Aufrufweg **Formular → Hook → Preload/IPC → Dispatcher → Handler → Service → Repository → SQLite**. Bei der Notenerfassung übermittelt die vorhandene Seite beispielsweise die Formulardaten über `useCreateGrade` beziehungsweise `useUpdateGrade`. Der generierte Hook bildet die Eingabe auf den IPC-Vertrag ab. Nach Prüfung und Ausführung gelangt das Ergebnis über den Dispatcher und den API-Client zurück zum Hook. Die Oberfläche aktualisiert anschließend die Abfragen für Notenliste und Statistik. [31]

Diese Grundstruktur kann für weitere CRUD-Bereiche erneut angewandt werden. Dabei sind jeweils die zugehörigen Vertragsdefinitionen, fachlichen Regeln, Datenzugriffe und Berechtigungen zu berücksichtigen. Die Wiederholung betrifft die Verteilung der Verantwortlichkeiten und den Kommunikationsweg; die Berechtigungsregeln gemeinsamer Gruppenressourcen sind gesondert von der persönlichen Datenzuordnung zu prüfen. Die Struktur konkretisiert damit A2 und das in A8 geforderte Erweiterungsmuster.

`scripts/grades/grades.test.ts` verbindet den generierten Client mit Dispatcher, Handlern, Service, Repositories und einer SQLite-Datenbank im Arbeitsspeicher. Geprüft werden Sitzung, persönliche Datenzuordnung, Änderungen, Aktivitäten und Statistik. Der direkte Dispatcher-Aufruf ersetzt die native IPC-Kommunikation. Der Test prüft damit die Fachlogik samt Datenzugriff; Oberfläche und Electron-Laufzeit werden gesondert bewertet. [31]

5.2.11 Lokaler Dateiimport am Beispiel iCal

Das Importmuster wird am iCal-Import aus Commit `fe08220` beschrieben. Im Web-Prototyp übermittelte die Kalenderseite die gewählte Datei als `FormData` mit `fetch` an `/api/events/import-ical`. Multer nahm den Upload entgegen; der Route-Handler interpretierte die Kalenderdaten und schrieb die Termine in die Datenbank. [6], [32]

In der Desktop-Anwendung wurde diese Upload-Verarbeitung in eine lokale Importoperation überführt. Hierfür wurde der OpenAPI-Vertrag um `importEventsIcal` und das Ergebnisschema `EventIcalImportResult` ergänzt. Anschließend wurden Client, Hook und Validierungsschemata neu erzeugt. Die Operation besitzt keine Dateipfad- oder Inhaltsparameter: Der Renderer löst über `useImportEventsIcal` die Auswahl aus, während der Main-Prozess die Datei ermittelt und verarbeitet. Damit wird die in Abschnitt 5.2.4 beschriebene zentrale Vertragspflege an einer zusätzlichen Schnittstelle angewandt. [32]

Die Dateiauswahl übernimmt `NativeIcalFilePicker` im Main-Prozess. Er verwendet `dialog.showOpenDialog` mit einem Filter für ICS-Dateien und bindet den Dialog, sofern verfügbar, an das aus dem IPC-Absender ermittelte Anwendungsfenster. [32] Electron liefert aus diesem Dialog die ausgewählten Pfade beziehungsweise einen Abbruchzustand zurück. [33] Der Pfad bleibt innerhalb der nativen Auswahl und wird nicht vom Renderer als frei wählbares Argument übergeben. Die Implementierung prüft die Endung `.ics`, den regulären Dateityp und eine Größenobergrenze von 10 MiB vor sowie nach dem Einlesen. Anschließend stellt sie den Inhalt als UTF-8-Text für die Verarbeitung bereit. [32]

Die Interpretation der Kalenderdaten wurde in `src/main/services/events-ical.parser.ts` ausgelagert. Das Modul verwendet weiterhin `node-ical`, prüft die grundlegende Kalenderstruktur und bildet verwertbare `VEVENT`-Einträge auf interne Termindaten ab. Fehlende Titel oder ungültige Datumswerte führen zum Überspringen der betreffenden Einträge; diese werden gezählt. Aus der ursprünglichen Verarbeitung wurden unter anderem die Ableitung des Termintyps aus dem Titel und eine Standarddauer bei fehlendem Enddatum übernommen. Der Parser erzeugt die Termindaten ohne Benutzerkennung und ohne Datenbankzugriff. [6], [32]

`EventsService.importEventsIcal` übernimmt die fachliche Verarbeitung der Parserausgabe. Der Service ergänzt die aus der Sitzung stammende Benutzerkennung, übergibt die Datensätze an `EventsRepository.createManyEvents` und veranlasst den Aktivitätseintrag zum Import. Dateiauswahl, Formatinterpretation, fachliche Zuordnung und Datenzugriff erhalten dadurch getrennte Zuständigkeiten. Beim Start wird der native Picker als Abhängigkeit an `createEventsHandlers` übergeben. Die Schnittstelle `IcalFilePicker` ermöglicht zugleich, die Dateiauswahl in Tests durch eine geeignete Implementierung zu ersetzen. [32]

Für den Import wurde eine ausdrückliche Ergebnisbehandlung eingerichtet. Ein erfolgreicher Import liefert `status: imported` sowie `importedCount` und `skippedCount`. Ein abgebrochener Dateidialog liefert `status: cancelled` mit beiden Zählwerten `0` und verändert den Datenbestand nicht. Ungültige Dateiauswahl oder Kalenderinhalte werden im Handler über die vorgesehenen Fehlerklassen in `VALIDATION_ERROR` und die deklarierte Fehlerantwort überführt. Diese Ergebnisse durchlaufen denselben Dispatcher und API-Client wie die CRUD-Operationen. [32]

In der Kalenderseite ersetzt der generierte Mutation-Hook den direkten HTTP-Aufruf und das HTML-Dateifeld. Während der Operation wird der Importknopf über `isPending` deaktiviert. Nach einem erfolgreichen Import wird die Terminanfrage invalidiert; die Oberfläche zeigt die Anzahl importierter und gegebenenfalls übersprungener Einträge an. Ein Abbruch beendet den Ablauf ohne Erfolgsmeldung, während ein Fehler gesondert angezeigt wird. Die ursprüngliche Upload-Oberfläche wird damit zu einem Auslöser für die im Main-Prozess ausgeführte lokale Importfunktion. [32]

Die Termintests prüfen Parserfälle, Benutzerzuordnung, Aktivitätseintrag, Abbruch und ungültige Kalenderinhalte. Ein ersetzter `IcalFilePicker` liefert vorgegebene Inhalte oder einen Abbruchzustand. So werden Handler, Parser, Service und Repository unabhängig vom Dateidialog geprüft. Native Dialoginteraktion und die Prüfungen beim Lesen realer Dateien benötigen einen gesonderten Nachweis. [32]

Der CSV-Notenimport aus Commit `d37f1b5` verwendet dieselbe Aufteilung in nativen Dateipicker, Import-Handler, Service und Repository. Die Formatverarbeitung übernimmt hier der eigene Parser `grades-csv.parser.ts`, der Spalten erkennt und unter anderem Trennzeichen, zitierte Felder und Dezimalkommas verarbeitet. Die Vertragsoperation `importGradesCsv` und der erzeugte Hook liefern entsprechend den Import- oder Abbruchstatus sowie Zählwerte. CSV zeigt damit, dass das beschriebene Importmuster sowohl eine eingebundene Parserbibliothek als auch einen eigenen Parser aufnehmen kann. [34]

5.2.12 Abschluss der Offline-Kernanwendung und Übergang zu Phase 2

Mit der lokalen Datenhaltung, der übernommenen Oberfläche, den generierten Aufrufhilfen und den implementierten Verarbeitungspfaden ist die erste Migrationsphase abgeschlossen. Die persönlichen Kernfunktionen werden innerhalb der Desktop-Anwendung ausgeführt: Lokale Anmeldung, Dashboard, Kalender, Noten und Stundenplan greifen über registrierte Handler, Services und Repositories auf SQLite zu; Dateiimporte werden im Main-Prozess verarbeitet. Die für Studiengänge und Stundenplanvorlagen benötigten Ausgangsdaten werden mit den Migrationen bereitgestellt. Für diese Abläufe wird kein Web-Backend benötigt. [35]

Der Kontrollpunkt umfasst den Start der gebauten Anwendung ohne Netzwerk, die Kernabläufe und die Persistenz nach einem Neustart (A1 und A5). Schichtentrennung (A2) und lokale Kommunikation (A4) werden anhand ihrer Umsetzung geprüft. Die Fachlogiktests ergänzen diesen Nachweis; Laufzeit und Persistenz sind in Kapitel 6 gesondert zu bewerten. Wiederverwendung und Erweiterungsmuster unterstützen die Bewertung von A3 und A8.

Phase 2 baut auf dieser lokalen Kernanwendung auf. Sie ergänzt die Synchronisierung gemeinsamer Aufgaben sowie den Abruf externer Katalogdaten. Damit werden zusätzliche Kommunikationspfade eingeführt, während die persönlichen Daten und ihre Verarbeitung lokal bleiben.

5.3 Phase 2: Netzwerkanbindungen und Local-First-Synchronisierung

Die zweite Phase ergänzt gezielt Funktionen, die eine Verbindung zu entfernten Diensten benötigen. Die lokale Kernanwendung bleibt dabei die Grundlage. Local-First bedeutet für den hier untersuchten Aufgabenbereich, dass eine Änderung zunächst lokal verarbeitet werden kann und ihre Übermittlung als nachgelagerter Vorgang erfolgt. Die Phase verfolgt keinen Anspruch auf vollständige Umsetzung sämtlicher Local-First-Eigenschaften.

5.3.1 Auswahlkriterien und betrachtete Synchronisierungslösungen

Die Toolauswahl knüpfte an A3 an: Die in Phase 1 eingerichtete Schichtentrennung, TypeScript und Drizzle sollten möglichst weitgehend weiterverwendet werden. Zugleich verlangte A6 die Synchronisierung gemeinsamer Aufgaben nach lokaler Verarbeitung. Eine Eigenentwicklung hätte zusätzlich

Verfahren für ausstehende Änderungen, Wiederholungen nach Verbindungsunterbrechungen, den Abgleich der Datenbestände und Konfliktbehandlung erfordert. Um den Umfang der Arbeit auf die Migration und ihre Muster zu begrenzen, wurde deshalb nach einer vorhandenen Synchronisierungslösung gesucht.

Für die Vorauswahl waren insbesondere lokale SQLite-Verarbeitung, eine Drizzle-Anbindung, die Ausführung im Electron-Main-Prozess und die Anbindung eines entfernten Backends relevant. In die engere Auswahl kamen PowerSync und Turso. PowerSync stellt mit `@powersync/drizzle-driver` eine Anbindung für typisierte Drizzle-Abfragen bereit. [36] Für Turso besteht eine Drizzle-Anbindung über den bereits verwendeten libSQL-Treiber. [37] Beide Ansätze boten damit Anknüpfungspunkte an die lokale Datenhaltung aus Phase 1.

Die Kandidaten übernehmen unterschiedliche Aufgaben. PowerSync bildet eine Synchronisierungsschicht zwischen einer Backend-Datenbank und lokalem SQLite; für die Node.js-Laufzeit steht ein eigenes SDK bereit. [38] Turso verbindet eine SQLite-kompatible Datenbank mit einem Cloud-Dienst und Replikationsmöglichkeiten. Die dokumentierten Embedded Replicas unterstützen lokale Datenhaltung und einen Abgleich mit Turso Cloud. [39] Tabelle 5.4 fasst die für die Vorauswahl relevanten Anknüpfungspunkte zusammen.

Aspekt	**PowerSync**	**Turso/libSQL**
Rolle	Synchronisierungsschicht vor einer Backend-Datenbank.	Datenbank und Cloud-Dienst mit lokalen Replikaten.
Drizzle-Anbindung	Wrapper und Ableitung des lokalen Schemas über `@powersync/drizzle-driver`.	Datenzugriff über `drizzle-orm/libsql` und den libSQL-Client.
Entfernte Datenhaltung	Anbindung von Supabase/ PostgreSQL im gewählten Entwurf.	Abgleich mit der Datenbank in Turso Cloud.

Tabelle 5.4: Dokumentationsbasierte Gegenüberstellung der für die Vorauswahl betrachteten Integrationsansätze. [36], [37], [39], [40]

5.3.2 Entscheidung für PowerSync und Supabase

Für den Demonstrator wurde eine Architektur mit entferntem Backend gewählt, das den gemeinsamen Datenbestand der Gruppen hält. Ein Peer-to-Peer-Ansatz wäre grundsätzlich eine andere Entwurfsrichtung; er wurde in dieser Arbeit nicht weiterverfolgt. Auch für das Backend sollten vorhandene Dienste verwendet werden, um die Infrastrukturentwicklung innerhalb des festgelegten Untersuchungsumfangs zu halten.

Die Wahl fiel auf Supabase, das eine PostgreSQL-Datenbank sowie Authentifizierung mit SDKs und API-Endpunkten bereitstellt. [41], [42] Ausschlaggebend war seine dokumentierte Integration mit PowerSync: Ausgewählte PostgreSQL-Daten werden in lokale SQLite-Datenbanken übertragen; lokale Änderungen werden gespeichert, in eine Upload-Warteschlange aufgenommen und bei verfügbarer Verbindung über einen Connector an das Backend übermittelt. Sync Streams legen fest, welche Daten an die jeweiligen Clients übertragen werden. [40]

Diese Kombination passte zum ausgewählten Gruppenfeature: Unterschiedliche Benutzer sollen auf gemeinsame Aufgaben zugreifen und Änderungen zunächst lokal ausführen können. PowerSync verband dafür die lokale SQLite-Verarbeitung und die Drizzle-Anbindung mit Supabase als entferntem Backend. Die Anwendung musste weiterhin ihre Datenschemata, den Upload-Connector und die

fachlichen Zugriffsregeln implementieren. Der Einsatz der Dienste begrenzte damit die zusätzlich zu entwickelnde Synchronisierungsinfrastruktur.

Die Entscheidung gegen Turso beruht auf dieser projektbezogenen Passung. Auch Turso ermöglicht Synchronisierung und Offline-Schreibzugriffe; die Turso-Sync-Dokumentation beschreibt hierfür lokale Änderungen mit späterem `push()` und den Empfang entfernter Änderungen über `pull()`. [43] Für den gewählten Supabase-Entwurf bot PowerSync eine unmittelbar dokumentierte Verbindung von Authentifizierung, Backend-Datenbank und gezielter Datenverteilung. [40] Die Auswahl begründet daher die verwendete Kombination, ohne eine allgemeine Überlegenheit gegenüber Turso zu behaupten.

5.3.3 Vorprüfung der Integration und Grenzen der Wiederverwendung

Die praktische Integrationsprüfung ist für PowerSync in zwei begrenzten Prototypen dokumentiert. Der erste prüfte die Node-SDK-Ausführung im Electron-Main-Prozess, lokale Persistenz samt Upload-Warteschlange, Schemaerweiterung, Geräteabgleich und Paketierung. Der zweite ergänzte den Drizzle-Wrapper und prüfte typisierte CRUD-Abfragen sowie die Abbildung von Datentypen. Die Projektberichte dokumentieren erfolgreiche lokale Prüfungen und einen Abgleich über Supabase und PowerSync. Sie stützten die Entscheidung, zunächst einen ausgewählten Kollaborationsbereich zu integrieren. Diese Vorprüfung untersucht die gewählte Kombination; sie ist kein praktisch durchgeführter Vergleich beider Kandidaten. [44]

Die Drizzle-Anbindung ermöglicht Wiederverwendung auf der Ebene der Typdefinitionen, Abfragen und Repository-Muster. Im Prototyp wird das lokale PowerSync-Schema mit `DrizzleAppSchema` aus geeigneten Drizzle-Definitionen abgeleitet; `wrapPowerSyncWithDrizzle` stellt den typisierten Datenzugriff bereit. [44]

Für synchronisierte Tabellen übernimmt PowerSync die Verwaltung der lokalen Datenbankstruktur. Drizzle Kit bleibt für die persönliche Datenbank aus Phase 1 zuständig; serverseitige PostgreSQL-Migrationen und die Sync-Stream-Konfiguration werden gesondert gepflegt. [44] Damit konnte Drizzle als typisierte Datenzugriffsschicht erhalten bleiben, während die Verwaltung des synchronisierten Datenbestands und seiner Identitäten gezielt angepasst wurde. Dieser Unterschied bestimmt die nachfolgende Umsetzung der Gruppenfunktionen.

5.3.4 Architektur mit zwei lokalen Datenhaltungsbereichen

Mit PowerSync und Supabase erweitert sich die lokale Architektur um entfernte Dienste. Der Renderer verwendet weiterhin die begrenzte Preload-/IPC-Schnittstelle; Datenzugriff und Dienstanbindung werden im Main-Prozess koordiniert. Persönliche Funktionen und ursprüngliche lokale Gruppen verbleiben in der SQLite-Datenbank aus Phase 1. Der neue Kollaborationsbereich verwendet eine weitere lokale SQLite-Datenbank, deren Struktur und Synchronisierung PowerSync verwaltet. Für jede entfernte Benutzeridentität wird eine eigene Datei geöffnet. Es bestehen damit zwei lokale Datenhaltungsbereiche; hinzu kommt der gemeinsame PostgreSQL-Datenbestand bei Supabase. [45]

Abbildung 5.2 zeigt die getrennten Kommunikationswege. Aufgabenänderungen werden zunächst lokal gespeichert und über den Upload-Connector an die Supabase-API übermittelt. PowerSync überträgt die anhand der Sync Streams ausgewählten Backend-Daten zurück in den lokalen Kollaborationsbestand. Der Upload führt somit über die Anwendungs-API, während der Download durch den PowerSync-Dienst erfolgt. [40], [45]

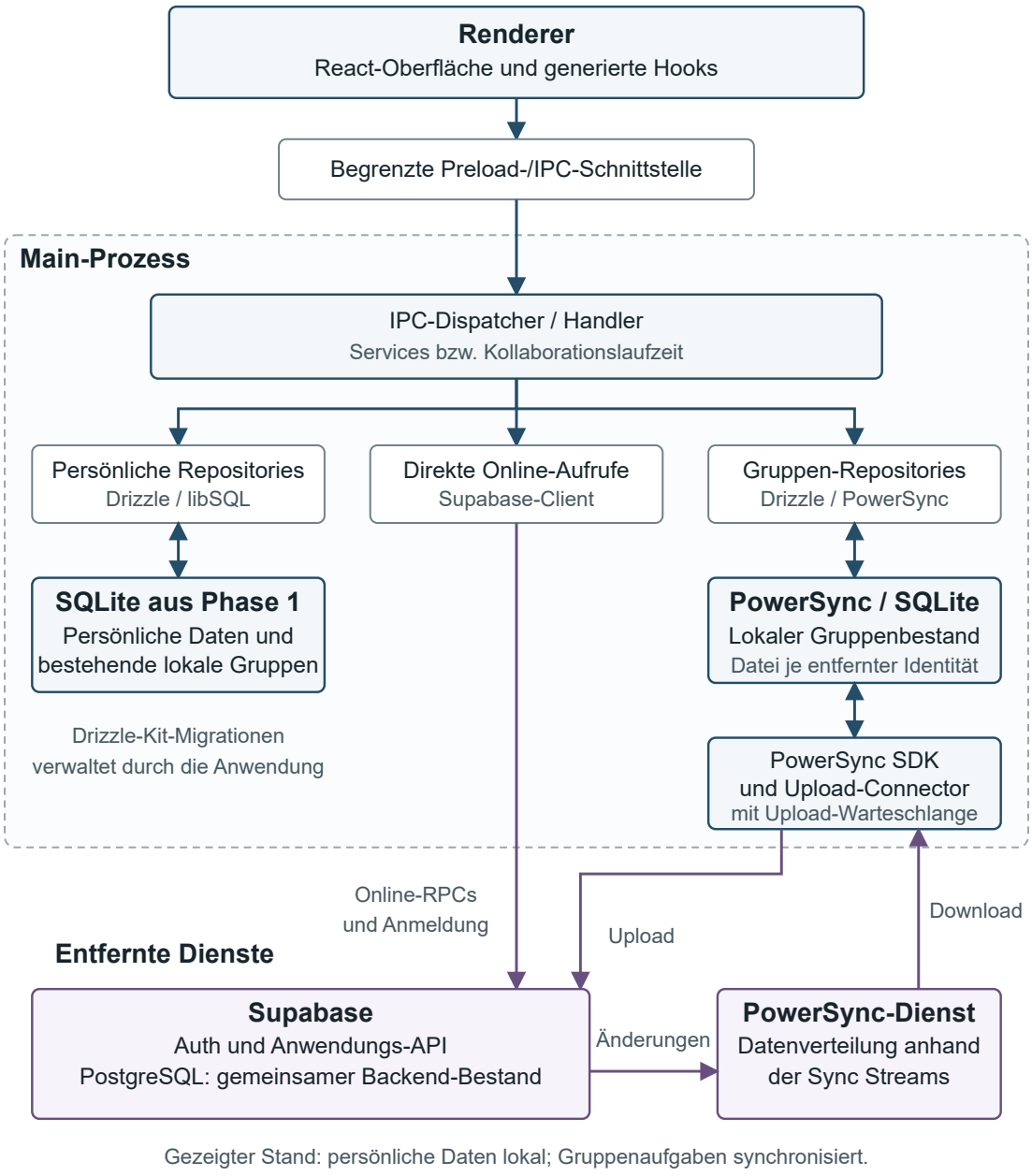


Abbildung 5.2: Architektur nach Ergänzung des synchronisierten Gruppenbereichs. Dargestellt sind die beiden lokalen Datenhaltungsbereiche sowie direkte Online-Aufrufe, Upload und Download. Der separate Katalogabruf ist hier nicht dargestellt. Eigene Darstellung nach [45] und [40].

Gruppenanlage, Profilsuche und Einladungen verwenden unmittelbare Online-Aufrufe. Die entfernte Sitzung wird im Demonstrator nur im Arbeitsspeicher gehalten; nach einem Neustart ist eine erneute Anmeldung erforderlich. Die lokale Speicherung von Aufgaben bedeutet deshalb keine uneingeschränkte Offline-Verfügbarkeit aller Gruppenoperationen. [45]

5.3.5 Technische Integration der Kollaborationsmodule

Die Integration begann mit der Abstimmung des Datenmodells zwischen Supabase und lokalem PowerSync. In Supabase wurden die fünf Tabellen des Gruppenbereichs durch versionierte SQL-Migrationen angelegt. Datensatz- und Benutzerkennungen werden dort als UUIDs geführt; das lokale Drizzle-Schema bildet sie als Textfelder ab. Neue Aufgaben erhalten ihre UUID bereits beim lokalen Einfügen. Zeitangaben werden lokal als ISO-Strings und boolesche Werte als SQLite-Integer repräsentiert. `schema.ts` definiert diesen lokalen Tabellenbestand und leitet daraus mit `DrizzleAppSchema` das PowerSync-Schema ab. Die neuen Kennungen unterscheiden sich damit von den numerischen Kennungen des Ausgangsmodells. [46]

Anschließend wurden die Tabellen in die PostgreSQL-Publication `powersync` aufgenommen und für deren Replikationsrolle lesbar gemacht. Die Verbindung des PowerSync-Dienstes zu Supabase umfasst die Datenbankanbindung und die Anerkennung der Supabase-Anmeldetoken. [40] Die Datenverteilung wurde in `sync-streams-v4.yaml` beschrieben und als Dienstkonfiguration bereitgestellt. Die Abfragen berücksichtigen Gruppenmitgliedschaften sowie Absender und Empfänger von Einladungen. Diese Datei steuert den Download; die für Schreibzugriffe geltenden Berechtigungen sind zusätzlich im Backend definiert. [46] Die entfernten SQL-Migrationen und Sync Streams ergänzen damit das lokale Anwendungsschema und werden getrennt von der Desktop-Paketierung gepflegt.

Ergänztes Paket	**Aufgabe im Demonstrator**
`@powersync/node`	Lokale PowerSync-Datenbank, Warteschlange und Synchronisierungsverbindung.
`@powersync/drizzle-driver`	Ableitung des Client-Schemas und typisierter Drizzle-Zugriff.
`@supabase/supabase-js`	Anmeldung, direkte RPC-Aufrufe und Upload an die Supabase-API.
`better-sqlite3`	Nativer SQLite-Treiber für die gewählte PowerSync-Implementierung.

Tabelle 5.5: Für den Kollaborationsbereich ergänzte Laufzeitabhängigkeiten und ihre Aufgaben. [46]

Für die neue Datenbank wurde `better-sqlite3` gewählt, während die Datenbank aus Phase 1 weiterhin über libSQL angebunden bleibt. Das PowerSync-Node-SDK unterstützt verschiedene SQLite-Treiber; der Wechsel beschreibt somit die konkrete Implementierung des Demonstrators. [38] `database.ts` öffnet die PowerSync-Datenbank und stellt mit `wrapPowerSyncWithDrizzle` den Zugriff für die Repositories bereit. Die Ausführung verwendet einen eigenen Worker, der über `powersync.worker?nodeWorker` in den Electron-Build eingebunden wird. [46]

Auch die Paketierung wurde angepasst: In `electron-builder.yml` werden `node_modules/@powersync/node/**` und `node_modules/better-sqlite3/**` unter `asarUnpack` aufgeführt. Der Worker ermittelt die native PowerSync-Erweiterung im Verzeichnis `app.asar.unpacked`. Diese Anpassung adressiert das im Prototyp beobachtete Laden der SQLite-Erweiterung unter Windows. Das vorhandene `postinstall`-Skript mit `electron-builder install-app-deps` dient dem Aufbau nativer Abhängigkeiten für die Electron-Laufzeit. [44], [46] Die Electron-Dokumentation beschreibt die erforderliche Entpackung bei APIs, die reale Dateipfade für native Module benötigen. [47]

Die Anwendungsmodule erhielten getrennte Zuständigkeiten. `CollaborationRuntime` koordiniert Sitzung, Lebenszyklus und fachliche Operationen. Bei der Anmeldung öffnet sie eine zur entfernten

Benutzerkennung gehörende Datenbank und verbindet diese mit dem Connector. Der gekürzte Auszug in Listing 5.5 zeigt diesen Schritt. Die Repositories lesen den lokalen Gruppenbestand und schreiben Aufgabenänderungen. `online-gateway.ts` führt die unmittelbar netzwerkabhängigen Gruppen- und Einladungsoperationen aus. Der Connector liefert über `fetchCredentials` das Anmeldetoken für PowerSync und übermittelt mit `uploadData` die ausstehenden Aufgabenänderungen an Supabase. [46]

```
const userId = await connection.connector.getUserId()
const directory = join(this.userDataDirectory, 'collaboration')
database = await openCollaborationDatabase(
  directory,
  `collaboration-${userId}.db`
)
await database.powerSync.connect(connection.connector)
```

Listing 5.5: Öffnen und Verbinden der benutzerbezogenen PowerSync-Datenbank; gekürzter Auszug aus `CollaborationRuntime.signIn`. [46]

Die Preload-Ebene stellt nun zwei begrenzte Aufrufwege bereit. Über `window.apiTransport.invoke` verwendet die Kernanwendung weiterhin den Kanal `api:invoke`. Für die Kollaboration wurde `window.collaborationTransport.invoke` mit dem Kanal `collaboration:invoke` ergänzt. Der neue Modulvertrag liegt in `src/shared/collaboration/ipc.ts` und wird im betrachteten Stand eigenständig typisiert, statt durch den OpenAPI-Generator erzeugt zu werden. Der zugehörige Main-Handler prüft Absender, Operation und Eingaben mit Zod und berücksichtigt die lokale Sitzung. [46] Die beiden IPC-Pfade bilden die getrennte Integration der Funktionsbereiche ab. Zwei Datenbanktreiber erzwingen diese Aufteilung nicht: Die Schnittstellen bieten fachliche Operationen an, deren Datenzugriff hinter den Repositories liegt. Eine Zusammenführung der Datenbanken würde daher nicht automatisch die IPC-Verträge vereinheitlichen.

Abbildung 5.3 zeigt die Module und ihre Verbindungen. Sie ergänzt das Architekturbild aus Abbildung 5.2 um die konkrete Aufteilung der Verarbeitung und die beiden IPC-Pfade.

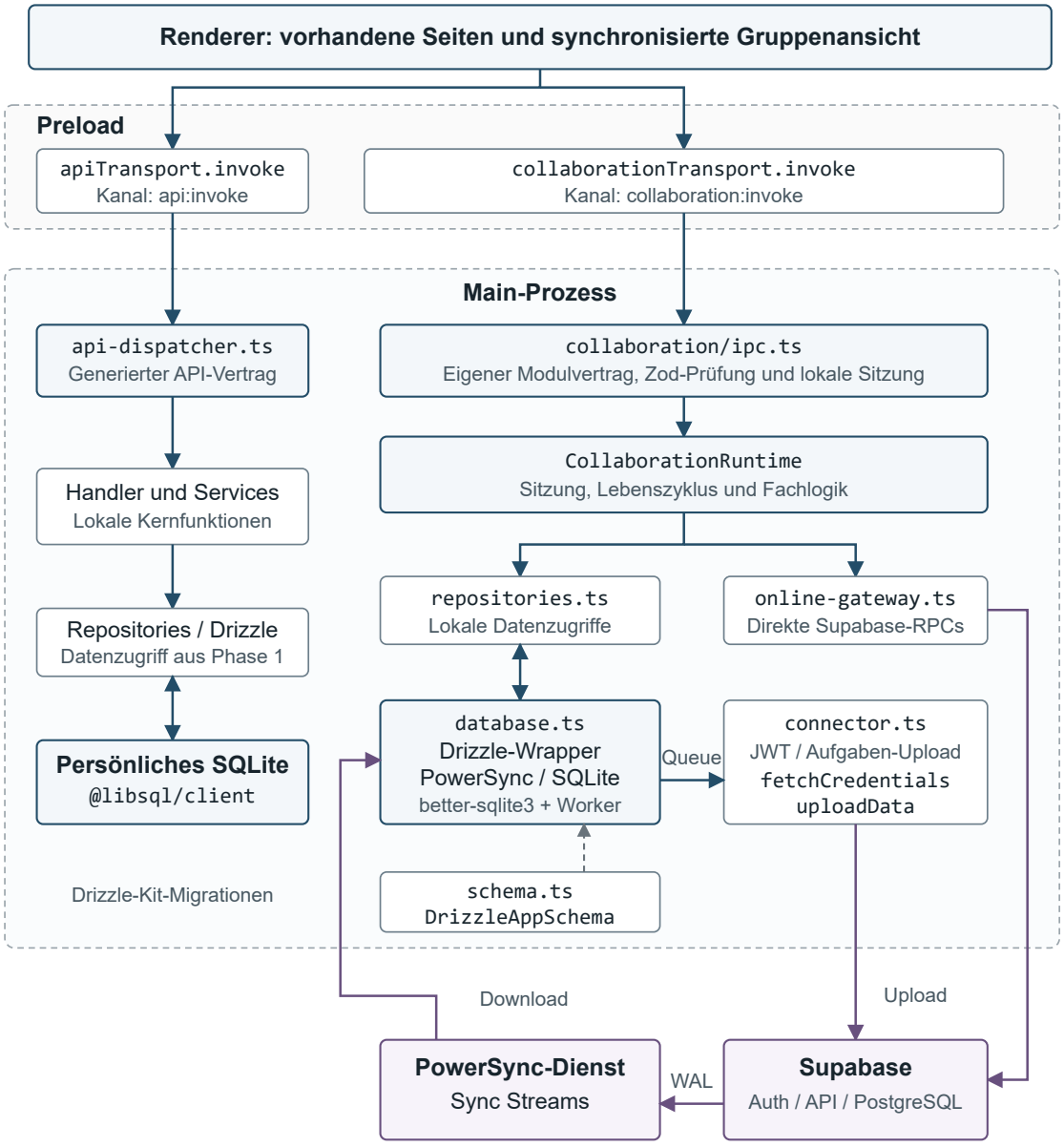


Abbildung 5.3: Modulaufteilung der Kollaborationsintegration neben der lokalen Kernanwendung. Blaue Pfeile zeigen lokale Aufruf- und Datenzugriffswege, violette Pfeile Dienstkommunikation; der gestrichelte Pfeil zeigt die Schemaeinbindung. Eigene Darstellung nach [46].

5.3.6 Anpassung des Einladungsablaufs und vorausgesetzte Konten

Der Ausgangsprompt benannte Gruppeneinladungen über Link oder E-Mail, ohne den Ablauf einer gezielten Benutzerauswahl innerhalb der Anwendung festzulegen. Im Web-Prototyp erzeugte die Gruppenroute einen Einladungstoken. Die Oberfläche bildete daraus eine URL und kopierte sie in die Zwischenablage; die Weitergabe an andere Personen erfolgte außerhalb der Anwendung. [6] Dieser Code beschreibt einen vorgesehenen Ablauf. Ein Nachweis seiner tatsächlichen Nutzbarkeit für die untersuchte Zusammenarbeit wurde in dieser Arbeit nicht geführt. Die Migration konnte deshalb nicht von einem bereits vollständig geprüften Kollaborationsprozess ausgehen.

Für den synchronisierten Gruppenbereich wurde ein anderer Einladungsablauf implementiert. Supabase Auth verwaltet die entfernten Benutzerkonten; die anwendungsbezogenen Namen und Kennungen liegen in einer zugeordneten `profiles`-Tabelle. Die Trennung zwischen Auth-Daten und über die Anwendungs-API zugänglichen Profilen entspricht dem von Supabase dokumentierten Ansatz. [48] Neu entwickelt wurde eine Suche nach Benutzer- oder Anzeigenamen. Die Funktion `search_invitable_profiles` steht dem angemeldeten Gruppeneigentümer zur Verfügung und liefert bis zu 20 passende, als auffindbar markierte Profile. Eigene Identität, vorhandene Mitglieder und Personen mit bereits ausstehender Einladung werden ausgeschlossen. Als Ergebnis werden Kennung, Benutzername und Anzeigename übermittelt. [45]

Die Auswahl eines Profils führt zu einer personenbezogenen Einladung über `send_group_invitation`. Die eingeladene Person erhält den Eintrag durch PowerSync und kann ihn mit `respond_group_invitation` annehmen oder ablehnen. Nach der Annahme entsteht die Mitgliedschaft, anhand derer die Gruppendaten übertragen werden. [45] Diese Benutzersuche und der zugehörige Einladungsablauf sind fachliche Anpassungen des Demonstrators. Sie gehen über den Austausch des Transports hinaus und konkretisieren die im Ausgangsartefakt nur teilweise festgelegte Zusammenarbeit.

Für die Fallstudie wurden die benötigten entfernten Testkonten vorab über das Supabase-Dashboard angelegt. Ihre E-Mail-Adressen und Passwörter mussten somit bereits vorliegen. Die Desktop-Anwendung enthält eine Anmeldung für diese Konten: Im Main-Prozess ruft der Connector `signInWithPassword` mit den eingegebenen Zugangsdaten auf. [45] Die Methode authentifiziert bestehende Konten und übernimmt keine Registrierung. [49] Die lokale Kontoverwaltung aus Phase 1 bleibt davon getrennt.

Eine Registrierung für das entfernte Backend, E-Mail-Bestätigung und Passwortzurücksetzung wurden in der Anwendungsoberfläche nicht implementiert. Der Untersuchungsaufbau setzt eingerichtete Konten voraus, um authentifizierte Gruppenzugriffe und die Synchronisierung gemeinsamer Aufgaben zu demonstrieren. Die Ausarbeitung eines vollständigen Kontolebenszyklus würde zusätzliche Produktfunktionen und Anforderungen einführen. Für diesen begrenzten Nachweis wurden sie daher nicht entwickelt; bei einer produktiven Anwendung wären sie anhand des vorgesehenen Benutzerzugangs gesondert festzulegen. Die Fallstudie bewertet damit die Migration unter dieser Voraussetzung und weist keine vollständige entfernte Benutzerverwaltung nach.

5.3.7 Schrittweise Überführung weiterer Funktionen

Die parallele Datenhaltung ermöglicht eine Migration nach Funktionsbereichen: Ein vorhandener Bereich bleibt zunächst bestehen, während ein ausgewählter Bereich mit dem neuen Datenzugriff umgesetzt wird. Dieses Vorgehen folgt dem Grundgedanken des Strangler-Fig-Patterns, bestehende Funktionen schrittweise durch neue Implementierungen abzulösen. [50] Im Demonstrator sind die beiden Gruppenbereiche jedoch noch getrennt; vorhandene Gruppendaten werden nicht automatisch übernommen. [45]

Das Terminbeispiel aus der Vorprüfung konkretisiert die erforderlichen Anpassungen. Die ursprüngliche `events`-Tabelle besitzt numerische, automatisch erzeugte Kennungen und lokale Benutzerkennungen. Für die separate Prototyptabelle `spike_events` wurden clientseitige UUIDs, eine Supabase-Benutzerzuordnung und geeignete Datentypabbildungen verwendet. Geprüft wurden typisierte CRUD-Zugriffe, Persistenz ausstehender Änderungen und der Abgleich zwischen Replikaten. Damit liegt ein Beispiel für die technische Überführung eines weiteren Datenbereichs vor; die Übernahme vorhandener Termine einschließlich ihrer Referenzen wurde damit noch nicht durchgeführt. [44] Weitere Bereiche würden dieselben Grundschritte durchlaufen: Schema und Identitäten anpassen, Daten und Referenzen zuordnen, Repositories anbinden und das Verhalten prüfen. Die Umsetzung

jedes weiteren CRUD-Pfads würde das bereits gezeigte Muster wiederholen und wird daher nicht vollständig ausgearbeitet.

Eine spätere Zusammenführung in einer PowerSync-verwalteten lokalen Datenbank wäre grundsätzlich möglich, ohne sämtliche Inhalte zu teilen. Persönliche Tabellen müssten dafür ausdrücklich als lokale Tabellen konfiguriert werden, etwa mit `localOnly: true`; ihre Änderungen gelangen dann nicht in die Upload-Warteschlange. [36], [51] Das bloße Weglassen eines Sync Streams reicht hierfür nicht aus, da Streams den Download steuern. Benutzerzuordnung und Zugriff ohne entfernte Anmeldung wären bei einer solchen Zusammenführung weiterhin zu erhalten. Erst nach Ablösung aller bisherigen libSQL-Zugriffe könnte `@libsql/client` entfallen. Dazu wären die betroffenen Repositories, Datenbankinitialisierung und Schemaverwaltung zu überführen. Im gewählten Entwurf bliebe `better-sqlite3` als lokaler Treiber erhalten. Eine solche vollständige Ablösung wurde nicht geprüft; sie beschreibt eine mögliche Weiterentwicklung, nicht den erreichten Projektstand.

5.3.8 Verantwortung für Schemaänderungen und Updates

Die Einführung der Synchronisierung verändert auch die Verantwortung für spätere Schemaänderungen. In Phase 1 verwaltet die Desktop-Anwendung ihre Datenbank durch mitgelieferte Drizzle-Migrationen. Für synchronisierte Funktionen sind nun das entfernte PostgreSQL-Schema, die PowerSync-Datenverteilung und das lokale Anwendungsschema aufeinander abzustimmen. Die Supabase-Datenbank erhält eigene versionierte SQL-Migrationen. [52] Das Desktop-Update ist damit Bestandteil einer koordinierten Bereitstellung über mehrere Komponenten.

Auf Clientseite wird das mit `DrizzleAppSchema` abgeleitete Schema beim Öffnen der PowerSync-Datenbank an das SDK übergeben. [46] Im verwendeten Standardmodell speichert PowerSync Daten als JSON und stellt sie über SQLite-Views entsprechend dem lokalen Schema bereit. Eine aktualisierte Schemadefinition wird beim Start der neuen Anwendungsversion angewandt; für diese Views werden keine zusätzlichen Drizzle-SQL-Migrationsdateien benötigt. [53] Die Drizzle-Migrationen aus Phase 1 bleiben für die separate persönliche Datenbank zuständig.

Für eine additive Erweiterung, etwa eine neue synchronisierte Tabelle, ergibt sich folgender Bereitstellungsablauf:

1. Zunächst wird das Supabase-Schema erweitert. Dazu gehören die erforderlichen Zugriffsregeln und gegebenenfalls Anpassungen der Backend-Operationen.
2. Anschließend wird die Tabelle in die PostgreSQL-Publication aufgenommen und die Sync-Stream-Konfiguration angepasst. Die Verarbeitung der aktualisierten Konfiguration muss abgeschlossen sein, bevor die neue Funktion freigegeben wird.
3. Danach wird die Desktop-Version mit dem erweiterten Client-Schema und den zugehörigen Anwendungsfunktionen veröffentlicht.

Diese Reihenfolge entspricht dem von PowerSync beschriebenen Vorgehen für additive Änderungen. Umbenennungen, Löschungen oder Typänderungen können andere Übergangsschritte erfordern. [54]

Die Abstimmung muss auch ältere Clients berücksichtigen, die ein Update noch nicht installiert haben oder nach einer Offline-Phase ausstehende Änderungen übertragen. Bestehende Lese- und Schreibformate sind während des Übergangs kompatibel zu halten. PowerSync beschreibt dafür unter anderem versionierte Streams und Datentransformationen; deren konkrete Verwendung hängt von der Schemaänderung ab. [53] Ein vollständiger Ablauf mit unterschiedlichen Anwendungsversionen, Backend-Änderung und anschließender Desktop-Aktualisierung wurde im Demonstrator nicht geprüft. Dieser Abschnitt benennt die daraus abgeleitete Anforderung an spätere Updates und erweitert den bisherigen Implementierungsnachweis nicht.

5.3.9 Grenzen der untersuchten Synchronisierung

Die Untersuchung umfasst die lokale Verarbeitung gemeinsamer Aufgaben und deren späteren Abgleich. Eine vollständige Übernahme persönlicher Kontodaten auf andere Geräte oder gemeinsame Echtzeitbearbeitung von Dokumenten gehört gemäß Kapitel 4 nicht dazu. Der Replikatabgleich der Vorprüfung prüft die technische Datenübertragung, ersetzt jedoch keine vollständige Kontomigration. [44]

Der Web-Prototyp verarbeitete Änderungen unmittelbar über sein autoritatives Backend. Eine fachliche Zusammenführung konkurrierender Offline-Änderungen war dort nicht implementiert. [6] Local-First ergänzt diese Entwurfsfrage: Regeln für konkurrierende Bearbeitungen, Löschkonflikte oder manuelle Konfliktentscheidungen müssen aus den Anforderungen der jeweiligen Anwendung abgeleitet werden. Ihre umfassende Entwicklung würde neue Fachfunktionalität schaffen und geht über die hier demonstrierte Migration hinaus.

Auch der Demonstrator besitzt deshalb keine eigens entwickelte Konfliktzusammenführung. Sein Connector übermittelt Aufgabenänderungen mit `upsert`, `update` und `delete` an Supabase, ohne eine eigene Versionsprüfung. [45] Eine als Last Write Wins bezeichnete Regel müsste ausdrücklich festlegen, welche Reihenfolge maßgeblich ist; ihre fachliche Eignung ist hier nicht evaluiert. Die Begrenzung betrifft erweiterte Konfliktstrategien. Warteschlange, Wiederholungen und der Umgang mit abgelehnten Änderungen bleiben für die Migration relevant. PowerSync verwendet ein serverautoritatives Modell; aus einem lokalen Schreibabschluss folgt noch keine Bestätigung durch das Backend. [55]

5.3.10 Online-Abruf des Studienkatalogs

Der Studienkatalog ergänzt die Synchronisierung um einen einfacheren Netzwerkablauf: Fachliche Referenzdaten werden online bereitgestellt, in die lokale Datenbank übernommen und anschließend offline gelesen. Damit können Kataloginhalte unabhängig von einer neuen Anwendungsversion aktualisiert werden. Der Demonstrator ergänzt hierzu den in Phase 1 ausgelieferten Seed-Bestand um online abrufbare Beispieldaten; der vorhandene Grundbestand bleibt erhalten. [56]

Im Commit `9320911` wurde dafür eine Supabase Edge Function ergänzt, die Studiengänge und zugehörige Lehrveranstaltungen aus einer JSON-Datei über einen öffentlichen HTTP-Endpunkt bereitstellt. Dieser Endpunkt benötigt weder ein Benutzerkonto noch einen Zugriff auf die entfernte Datenbank. Änderungen am angebotenen Katalog erfordern eine erneute Bereitstellung dieser Datei mit der Funktion, jedoch keine neue Version der Desktop-Anwendung. Der bestehende OpenAPI-Vertrag wurde um `refreshStudyCatalog` und dessen Ergebnis mit den Anzahlen übernommener Studiengänge und Lehrveranstaltungen erweitert. Daraus wurden die IPC-Vertragsbestandteile und der React-Query-Hook erneut generiert. [56]

Beim Öffnen der Studienseite liest die Oberfläche zunächst den lokalen Bestand und löst zusätzlich die Aktualisierung aus. Der Main-Prozess lädt die JSON-Daten, validiert sie anhand der vorhandenen Feldschemata und übernimmt beide Datenbereiche gemeinsam in einer Transaktion. Anhand stabiler Kennungen werden neue Einträge eingefügt und vorhandene aktualisiert, sodass wiederholte Abrufe keine Duplikate erzeugen. In der Antwort fehlende Einträge werden nicht gelöscht. Nach erfolgreicher Übernahme werden die betroffenen Abfragen der Oberfläche erneut ausgeführt. [56]

Scheitern Abruf, Validierung oder Datenbankübernahme, bleibt der bisherige lokale Bestand nutzbar. Die Oberfläche zeigt den Fehler an und bietet einen erneuten Versuch an. Bereits übernommene Katalogdaten bleiben auch nach einem Neustart lokal verfügbar. Der untersuchte Stand ruft den Katalog bei jedem Seitenaufruf erneut ab; eine Versionsprüfung zur Erkennung neuer Katalogstände wurde nicht implementiert. Er zeigt damit das Muster eines Downloads mit dauerhafter lokaler

Datenhaltung. Die Desktop-Anwendung lädt dabei keine Katalogänderungen zum entfernten Dienst hoch. [56]

Der Kontrollpunkt dieser Phase betrifft insbesondere A6 und A8. Für Aufgaben sind der lokale Abschluss einer Änderung und ihre spätere Übernahme im Kollaborationsablauf getrennt zu beobachten. Bei unmittelbar netzwerkabhängigen Operationen sind Fehlermeldung und Verhalten ohne Verbindung auszuweisen. Bereits lokal verfügbare Funktionen dürfen durch den Ausfall einer zusätzlichen Online-Anbindung nicht unnötig blockiert werden.

5.4 Phase 3: Prüfung und Härtung der vervollständigten Anwendung

Nach der funktionalen Umsetzung wird die Gesamtheit der entstandenen Schnittstellen geprüft. Dazu gehören IPC-Eingänge, lokale Ressourcen, Navigationsmöglichkeiten und externe Kommunikationspfade. Die Electron-Sicherheitscheckliste bildet den festgelegten Bezugsrahmen. [3] Für jede Empfehlung werden Anwendbarkeit, erforderliche Maßnahme, konkreter Beleg und verbleibende Nachweisgrenze dokumentiert.

Die im Demonstrator vorgesehenen Maßnahmen betreffen unter anderem deaktivierte Node-Integration im Renderer, Context Isolation, Renderer-Sandboxing, eine restriktive Content Security Policy und die Begrenzung von Navigation sowie zusätzlichen Fenstern. IPC-Absender und fachliche Eingaben werden im Main-Prozess geprüft. Für externe Dienste sind die zugelassenen Verbindungsziele und ihre Transportkonfiguration zu kontrollieren. Paketbezogene Einstellungen sind am gebauten Artefakt zu prüfen, sofern sie Bestandteil der jeweiligen Empfehlung sind.

Der Kontrollpunkt ist A7. Ein Konfigurationseintrag, ein Unit-Test und eine beobachtete Laufzeitwirkung liefern unterschiedliche Nachweise und werden entsprechend ausgewiesen. Für SDK-interne Netzwerkpfade wird keine Prüfung behauptet, die tatsächlich nicht durchgeführt wurde. Die konsolidierte Projektmatrix dokumentiert insbesondere die begrenzte Bewertung der PowerSync-Redirectbehandlung; diese Nachweisgrenze ist bei der Ergebnisdarstellung zu übernehmen. [57]

5.5 Ausarbeitung der einzelnen Migrationsschritte

Aus dem dokumentierten Migrationsverlauf werden die Handlungsempfehlungen des Leitfadens abgeleitet. Jede Empfehlung verbindet einen konkreten Ausgangszustand mit der erforderlichen Transformation, den betroffenen Verantwortlichkeiten und einem Kontrollpunkt. Die Zuordnung zu A1–A9 macht nachvollziehbar, welches Ziel der jeweilige Schritt unterstützt.

Kapitel 6

Evaluation und Demonstration

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine metu degendae praesidia firmissima. – Filium morte multavit. – Si sine causa, nollem me ab eo delectari, quod ista Platonis, Aristoteli, Theophrasti orationis ornamenta neglexerit. Nam illud quidem physici, credere aliquid esse minimum, quod profecto numquam putavisset, si a Polyaeno, familiari suo, geometrica discere maluisset quam illum etiam ipsum dedocere. Sol Democrito magnus videtur, quippe homini erudito in geometriaque perfecto, huic pedalis fortasse; tantum enim esse omnino in nostris poetis aut inertissimae segnitiae est aut fastidii delicatissimi. Mihi quidem videtur,

inermis ac nudus est. Tollit definitiones, nihil de dividendo ac partiendo docet, non quo ignorare vos arbitrer, sed ut ratione et via.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine.

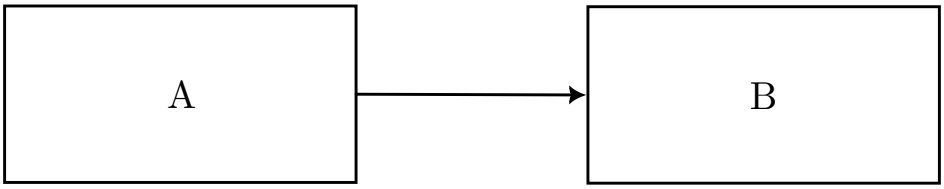


Abbildung 6.1: Example Figure

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine metu degendae praesidia firmissima. – Filium morte multavit. – Si sine causa, nollem me ab eo delectari, quod ista Platonis, Aristoteli, Theophrasti orationis ornamenta neglexerit. Nam illud quidem physici, credere aliquid esse minimum, quod profecto numquam putavisset, si a Polyaeno, familiari suo, geometrica discere maluisset quam illum etiam ipsum dedocere. Sol Democrito magnus videtur, quippe homini erudito in geometriaque perfecto, huic pedalis fortasse; tantum enim esse omnino in nostris poetis aut inertissimae segnitiae est aut fastidii delicatissimi. Mihi quidem videtur, inermis ac nudus est. Tollit definitiones, nihil de dividendo ac partiendo docet, non quo ignorare vos arbitrer, sed ut ratione et via.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine.

Kapitel 7

Diskussion

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine metu degendae praesidia firmissima. – Filium morte multavit. – Si sine causa, nollem me ab eo delectari, quod ista Platonis, Aristoteli, Theophrasti orationis ornamenta neglexerit. Nam illud quidem physici, credere aliquid esse minimum, quod profecto numquam putavisset, si a Polyaeno, familiari suo, geometrica discere maluisset quam illum etiam ipsum dedocere. Sol Democrito magnus videtur, quippe homini erudito in geometriaque perfecto, huic pedalis fortasse; tantum enim esse omnino in nostris poetis aut inertissimae segnitiae est aut fastidii delicatissimi. Mihi quidem videtur,

inermis ac nudus est. Tollit definitiones, nihil de dividendo ac partiendo docet, non quo ignorare vos arbitrer, sed ut ratione et via.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine.

Kapitel 8

Fazit und Ausblick

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine metu degendae praesidia firmissima. – Filium morte multavit. – Si sine causa, nollem me ab eo delectari, quod ista Platonis, Aristoteli, Theophrasti orationis ornamenta neglexerit. Nam illud quidem physici, credere aliquid esse minimum, quod profecto numquam putavisset, si a Polyaeno, familiari suo, geometrica discere maluisset quam illum etiam ipsum dedocere. Sol Democrito magnus videtur, quippe homini erudito in geometriaque perfecto, huic pedalis fortasse; tantum enim esse omnino in nostris poetis aut inertissimae segnitiae est aut fastidii delicatissimi. Mihi quidem videtur,

inermis ac nudus est. Tollit definitiones, nihil de dividendo ac partiendo docet, non quo ignorare vos arbitrer, sed ut ratione et via.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine.

Abbildungsverzeichnis

4.1 Ursprüngliche Webarchitektur des Studienplaners mit Laufzeitkommunikation und OpenAPI-basierter Codegenerierung; eigene Darstellung auf Grundlage des Ausgangsartefakts. 12
5.1 Vertragsbasierte Codegenerierung und Aufrufwege im Web-Prototyp und in der lokalen Electron-Kernanwendung. Gestrichelte Pfeile zeigen Generierung oder Einbindung, durchgezogene Pfeile Laufzeitkommunikation. Spätere Synchronisierungs- und Online-Pfade sind nicht dargestellt. 24
5.2 Architektur nach Ergänzung des synchronisierten Gruppenbereichs. Dargestellt sind die beiden lokalen Datenhaltungsbereiche sowie direkte Online-Aufrufe, Upload und Download. Der separate Katalogabruf ist hier nicht dargestellt. Eigene Darstellung nach [45] und [40]. 34
5.3 Modulaufteilung der Kollaborationsintegration neben der lokalen Kernanwendung. Blaue Pfeile zeigen lokale Aufruf- und Datenzugriffswege, violette Pfeile Dienstkommunikation; der gestrichelte Pfeil zeigt die Schemaeinbindung. Eigene Darstellung nach [46]. 37
6.1 Example Figure 43

Quellcodeverzeichnis

4.1 Exemplarischer Express-Route-Handler für die Abfrage anstehender Termine im Ausgangsartefakt. 13
5.1 Stundenplaneinträge im PostgreSQL-Schema des Ausgangsartefakts (Ausschnitt). 20
5.2 Stundenplaneinträge im SQLite-Schema des Demonstrators (Ausschnitt). 20
5.3 Drizzle-Kit-Konfiguration zur Erzeugung der SQLite-Migrationen. 21
5.4 Auslieferung der Migrationsdateien über zusätzliche Paketressourcen (Ausschnitt). 22
5.5 Öffnen und Verbinden der benutzerbezogenen PowerSync-Datenbank; gekürzter Auszug aus `CollaborationRuntime.signIn`. [46] 36

Tabellenverzeichnis

2.1 Example Dataset 5
4.1 Funktionaler Untersuchungsumfang auf Grundlage der ursprünglichen Erstellungs-Prompts. . 10
5.1 Phasen und Kontrollpunkte des fallbezogenen Migrationsleitfadens. 18
5.2 Wiederverwendete Vertragsgrundlage und Änderungen an ihrer Umsetzung. 26
5.3 Aufteilung der ursprünglichen Notenrouten auf Handler, Service und Repository. 29
5.4 Dokumentationsbasierte Gegenüberstellung der für die Vorauswahl betrachteten Integrationsansätze. [36], [37], [39], [40] 32
5.5 Für den Kollaborationsbereich ergänzte Laufzeitabhängigkeiten und ihre Aufgaben. [46] 35

Glossar

Referenzen

[1] Lovable, „App & Website Builder: Build Software in Minutes“. Zugegriffen: 6. Oktober 2026. [Online]. Verfügbar unter: https://lovable.dev/

[2] K. Peffers, T. Tuunanen, M. A. Rothenberger, und S. Chatterjee, „A Design Science Research Methodology for Information Systems Research“, *Journal of Management Information Systems*, Bd. 24, Nr. 3, S. 45–77, 2007, doi: 10.2753/MIS0742-1222240302.

[3] Electron, „Security“. Zugegriffen: 7. Oktober 2026. [Online]. Verfügbar unter: https://www.electronjs.org/docs/latest/tutorial/security

[4] Replit, „Web Apps“. Zugegriffen: 7. Oktober 2026. [Online]. Verfügbar unter: https://docs.replit.com/features/artifact-types/web-apps

[5] Lovable, „Connect to Supabase“. Zugegriffen: 7. Oktober 2026. [Online]. Verfügbar unter: https://docs.lovable.dev/integrations/supabase

[6] „Studienplaner: Replit-Ausgangsartefakt“. 2026.

[7] M. Kleppmann, A. Wiggins, P. van Hardenberg, und M. McGranaghan, „Local-first software: You own your data, in spite of the cloud“, in *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, 2019, S. 154–178. doi: 10.1145/3359591.3359737.

[8] Electron, „Process Model“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://www.electronjs.org/docs/latest/tutorial/process-model

[9] Tauri, „Process Model“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://v2.tauri.app/concept/process-model/

[10] Tauri, „Node.js as a sidecar“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://v2.tauri.app/learn/sidecar-nodejs/

[11] electron-vite, „Getting Started“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://electron-vite.org/guide/

[12] Electron Forge, „React with TypeScript“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://www.electronforge.io/guides/framework-integration/react-with-typescript

[13] Electron Forge, „Getting Started“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://www.electronforge.io/

[14] „StudyPlan: Lokale Datenhaltung und Schemamigrationen“. 2026.

[15] Electron, „app: getPath“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://www.electronjs.org/docs/latest/api/app#appgetpathname

[16] Drizzle Team, „SQLite column types“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://orm.drizzle.team/docs/sqlite/column-types

[17] Drizzle Team, „Drizzle Kit: push“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://orm.drizzle.team/docs/drizzle-kit-push

[18] Drizzle Team, „Drizzle Kit: generate“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://orm.drizzle.team/docs/drizzle-kit-generate

[19] SQLite, „Pragma statements supported by SQLite“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://www.sqlite.org/pragma.html

[20] „StudyPlan: Vertragsbasierte API-Generierung und IPC-Schnittstelle“. 2026.

[21] „StudyPlan: Übernahme und Anbindung der React-Oberfläche“. 2026.

[22] „StudyPlan: Lokaler Authentifizierungsablauf“. 2026.

[23] Electron, „Inter-Process Communication“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://www.electronjs.org/docs/latest/tutorial/ipc

[24] Electron, „contextBridge“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://www.electronjs.org/docs/latest/api/context-bridge

[25] OpenAPI Initiative, „OpenAPI Specification v3.1.0“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://spec.openapis.org/oas/v3.1.0.html

[26] electron-vite, „Configuring electron-vite“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://electron-vite.org/config/

[27] Microsoft, „TSConfig Reference: paths“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://www.typescriptlang.org/tsconfig/paths.html

[28] Tailwind Labs, „Installing Tailwind CSS with Vite“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://tailwindcss.com/docs/installation/using-vite

[29] Microsoft, „TSConfig Reference: noUnusedLocals“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://www.typescriptlang.org/tsconfig/noUnusedLocals.html

[30] Wouter Contributors, „Wouter: Customizing the location hook“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://github.com/molefrog/wouter#customizing-the-location-hook

[31] „StudyPlan: Migration der CRUD-Operationen für Noten“. 2026.

[32] „StudyPlan: Lokaler iCal-Import und seine Ergebnisbehandlung“. 2026.

[33] Electron, „dialog: showOpenDialog“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://www.electronjs.org/docs/latest/api/dialog#dialogshowopendialogwindow-options

[34] „StudyPlan: Lokaler CSV-Notenimport mit eigenem Parser“. 2026.

[35] „StudyPlan: Lokale Kernanwendung vor der Synchronisierungsintegration“. 2026.

[36] PowerSync, „PowerSync Drizzle Driver“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://www.npmjs.com/package/@powersync/drizzle-driver

[37] Drizzle Team, „Turso Cloud“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://orm.drizzle.team/docs/sqlite/connect-turso

[38] PowerSync, „Node.js Client SDK“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://docs.powersync.com/client-sdks/reference/node

[39] Turso, „Embedded Replicas“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://docs.turso.tech/features/embedded-replicas/introduction

[40] PowerSync, „Supabase + PowerSync“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://docs.powersync.com/integrations/supabase/guide

[41] Supabase, „Database“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://supabase.com/docs/guides/database/overview

[42] Supabase, „Auth“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://supabase.com/docs/guides/auth

[43] Turso, „Turso Sync: Usage“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://docs.turso.tech/sync/usage

[44] „StudyPlan: Vorprüfung von PowerSync und der Drizzle-Anbindung“. 2026.

[45] „StudyPlan: Architektur und Umsetzung des synchronisierten Gruppenbereichs“. 2026.

[46] „StudyPlan: Technische Integration von PowerSync und Supabase“. 2026.

[47] Electron, „ASAR Archives“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://www.electronjs.org/docs/latest/tutorial/asar-archives

[48] Supabase, „User Management“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://supabase.com/docs/guides/auth/managing-user-data

[49] Supabase, „JavaScript: signInWithPassword“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://supabase.com/docs/reference/javascript/auth-signinwithpassword

[50] M. Fowler, „Strangler Fig“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://martinfowler.com/bliki/StranglerFigApplication.html

[51] PowerSync, „Local-Only Usage“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://docs.powersync.com/client-sdks/advanced/local-only-usage

[52] Supabase, „Database Migrations“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://supabase.com/docs/guides/deployment/database-migrations

[53] PowerSync, „Implementing Schema Changes“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://docs.powersync.com/maintenance-ops/implementing-schema-changes

[54] PowerSync, „Deploying Schema Changes“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://docs.powersync.com/maintenance-ops/deploying-schema-changes

[55] PowerSync, „Writing Client Changes“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://docs.powersync.com/handling-writes/writing-client-changes

[56] „Online-Abruf und lokale Übernahme des Studienkatalogs“. 2026.

[57] „Konsolidierte Electron-Sicherheitsergebnismatrix“. 2026.

Anhang A

Ursprüngliche Erstellungs-Prompts

Dieser Anhang dokumentiert die beiden ursprünglichen Prompts, mit denen der Replit-Webprototyp Studienplaner erstellt und um Studiengruppen erweitert wurde. Die Texte werden im Wortlaut der vom Verfasser bereitgestellten Datei wiedergegeben. Rechtschreibung und Formulierungen bleiben erhalten; Überschriften, Absatzdarstellung und die Trennung der beiden Prompts wurden für die Dokumentation ergänzt.

Die Prompts belegen die Beschreibung von Funktionen und Nutzungsszenarien ohne Festlegung eines UI-Frameworks oder einer konkreten technischen Architektur. Sie enthalten zugleich Anforderungen an Anmeldung, Zugriffsschutz und die Trennung persönlicher und gemeinsamer Inhalte. Der ursprünglich gewünschte Funktionsumfang ist von dem in Kapitel 4 festgelegten Untersuchungsumfang zu unterscheiden; insbesondere begründet der im zweiten Prompt genannte Gerätewechsel keine Erweiterung des Migrations- oder Evaluationsumfangs dieser Arbeit.

Quelle ist die bereitgestellte Textdatei `Pasted-Hi-Ich-m-chte-eine-Web-App-f-r-mein-Studium-bauen-damit_1783021865134.txt`. Eine unveränderte Kopie ist im Thesis-Projekt unter `content/sources/original-creation-prompts.txt` abgelegt. Die Kennzeichnung „Prompt 2“ in der Quelldatei dient hier zur Aufteilung in die folgenden Abschnitte.

A.1 Prompt 1: Persönliche Studienorganisation

> Hi! Ich möchte eine Web-App für mein Studium bauen, damit ich endlich mal mein ganzes Chaos im Griff habe. Ich habe mir das so vorgestellt:
>
> Am Anfang soll man auf ein Dashboard kommen. Das ist quasi die Startseite, wo ich sofort sehe, wie mein aktueller Notenschnitt (GPA) ist, wie viele ECTS-Punkte ich insgesamt schon gesammelt habe und welche Fristen als Nächstes ablaufen. Da sollen auch ein paar schnelle Buttons sein, womit ich sofort eine neue Note oder einen neuen Termin eintragen kann, ohne erst mühsam in die Untermenüs zu klicken. Und eine kleine Liste mit den letzten Aktivitäten und den nächsten Events wäre super.
>
> Damit ich meine Termine im Blick habe, brauche ich einen Kalender. Ganz klassisch als Monatsübersicht. Wenn ich auf einen Tag klickt, soll sich so ein kleines Infofenster (Popover) öffnen, das mir alle Termine für den Tag anzeigt. Und wenn ich einen Termin anklicke, will ich die Details sehen und ihn auch löschen können. Die Termine müssen farblich sortiert sein, damit man sofort sieht, was Sache ist: Prüfungen und Deadlines in Rot, Anmeldefristen in Gelb, normale Vorlesungen in Blau und alles andere in Grün. Ganz wichtig: Ich will meine Termine von der Uni als iCal- oder ICS-Datei hochladen können, damit ich nicht alles einzeln abtippen

muss! Manchmal gehen Events auch über mehrere Tage, das muss der Kalender anzeigen können. Und eine normale Listenansicht, wo ich nach Wichtigkeit filtern kann, wäre auch cool.

Dann brauche ich einen Bereich für meine Noten. Das soll eine schöne Tabelle sein, wo alle meine Kurse mit den Noten und ECTS-Punkten drinstehen, am besten nach Semestern sortiert. Ich will Noten hinzufügen, bearbeiten und löschen können. Weil es nervt, alles einzutippen, wäre eine Funktion genial, wo ich eine CSV-Datei mit meinen Noten hochlade und die App schlau genug ist, zu erkennen, was in welcher Spalte steht. Der Notenschnitt muss sich dann automatisch anhand der ECTS-Gewichtung ausrechnen. Und die App muss aufpassen, dass ich nur Noten für Kurse eintragen kann, die auch wirklich zu meinem Studiengang gehören.

Für den Stundenplan stelle ich mir ein Raster vor: 5 Wochentage (Montag bis Freitag) und 6 feste Zeitblöcke am Tag. Die Zeiten an unserer Uni sind immer: 8:00-9:30, 9:45-11:15, 11:30-13:00, 14:00-15:30, 15:45-17:15 und 17:30-19:00. Man soll über ein Dropdown-Menü fertige Pläne von der Uni (die irgendwie aus den Daten der Uni-Website gezogen wurden) durchsuchen und auswählen können. Mit einem einfachen Plus- oder Minus-Button will ich dann Kurse zu meinem persönlichen Plan hinzufügen oder entfernen können. Man soll optisch sofort den Unterschied sehen zwischen den vorgeschlagenen Uni-Kursen und meinen selbst hinzugefügten. Und ein Button, um den ganzen Plan auf einmal zu löschen (nachdem man nochmal kurz gefragt wurde, ob man sich sicher ist), wäre top.

Ach ja, das Ganze muss natürlich sicher sein! Man muss sich registrieren und einloggen können, damit jeder nur seine eigenen Noten und Termine sieht. Und einen Ausloggen-Button brauche ich natürlich auch.

Ich habe leider überhaupt keine Ahnung von Programmieren oder wie man sowas baut, aber ich hoffe, du kannst mir daraus eine schicke, einfach zu bedienende App machen! Danke schonmal!

A.2 Prompt 2: Studiengruppen und Zusammenarbeit

Mir ist gerade noch etwas eingefallen: Ich würde die App gerne auch mit anderen aus meinem Semester benutzen können.

Vielleicht könnte man sogenannte Studiengruppen anlegen. Zum Beispiel für ein bestimmtes Modul, eine Lerngruppe oder die Bachelorarbeit. Eine Gruppe sollte einen Namen bekommen und man sollte andere Leute über einen Einladungslink oder per E-Mail hinzufügen können.

In einer Studiengruppe wäre es praktisch, wenn man gemeinsame Termine und Aufgaben verwalten könnte. Zum Beispiel:

Prüfungstermine
Abgabefristen
Lerntreffen
gemeinsame To-dos
wichtige Links oder Notizen
Bei Aufgaben sollte man ungefähr sehen können, worum es geht, wann sie fällig sind und wer sich darum kümmert. Man sollte Aufgaben auch als offen oder erledigt markieren können.

Zu gemeinsamen Terminen und Aufgaben sollten die Mitglieder Kommentare schreiben können. Außerdem wäre es hilfreich, wenn man sehen kann, wer etwas erstellt oder zuletzt geändert hat.

Die gemeinsamen Inhalte sollen nur für die Mitglieder der jeweiligen Gruppe sichtbar sein. Meine persönlichen Noten und privaten Termine sollen nicht automatisch mit anderen geteilt werden. Wenn ich etwas teilen möchte, sollte ich das selbst auswählen können.

Wenn jemand aus der Gruppe einen Termin oder eine Aufgabe ändert, sollten die anderen Mitglieder die Änderung sehen, wenn sie die App öffnen. Ich möchte die App außerdem mit meinem Benutzerkonto auch an einem anderen Gerät verwenden können, ohne dass meine Gruppen und Einträge dort fehlen.

Die Studiengruppen sollten sich möglichst einfach bedienen lassen und sich optisch gut in die bestehende Kalender- und Aufgabenansicht einfügen. Es wäre schön, wenn persönliche und gemeinsame Termine klar voneinander zu unterscheiden wären.